Database Management System (DBMS) – Installing and Using Microsoft SQL Server 2022

In this chapter we first build a solid theoretical understanding of what a Database Management System (DBMS) is, with a special focus on the key components of Microsoft SQL Server 2022. We then move into a complete, real-world, step-by-step installation carried out inside a Windows Server / Active Directory domain running on a Hyper-V virtual machine, followed by a hands-on lab covering 22 essential SQL commands. By the end of this chapter, even someone with absolutely no prior experience should be able to follow the screenshots and explanations here and reproduce the entire lab practical independently, without needing help from anyone else.

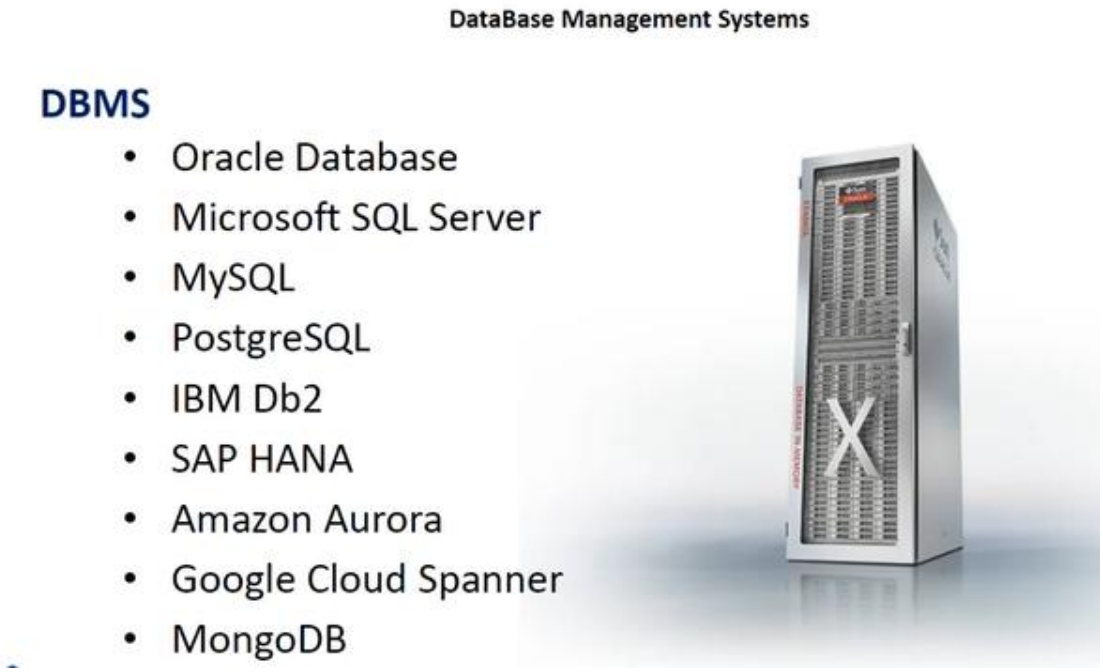
The chapter is organized into 5 parts:

- Part 01 – Theoretical introduction to DBMS and SQL Server
- Part 02 – Installing SQL Server 2022 – a complete hands-on lab
- Part 03 – 22 essential SQL commands explained in full detail
- Part 04 – Running those commands practically in SSMS
- Part 05 – Installing a bonus tool: ManageEngine Endpoint Central

Part 01 – Theoretical Introduction to DBMS and SQL Server

Before installing SQL Server, it is essential to understand its underlying concepts. Using the slide screenshots below, we will look in detail at the core components of a DBMS, the different editions of SQL Server, its engine, its management tools, its High Availability/Disaster Recovery mechanisms, its security features, and its Business Intelligence capabilities.

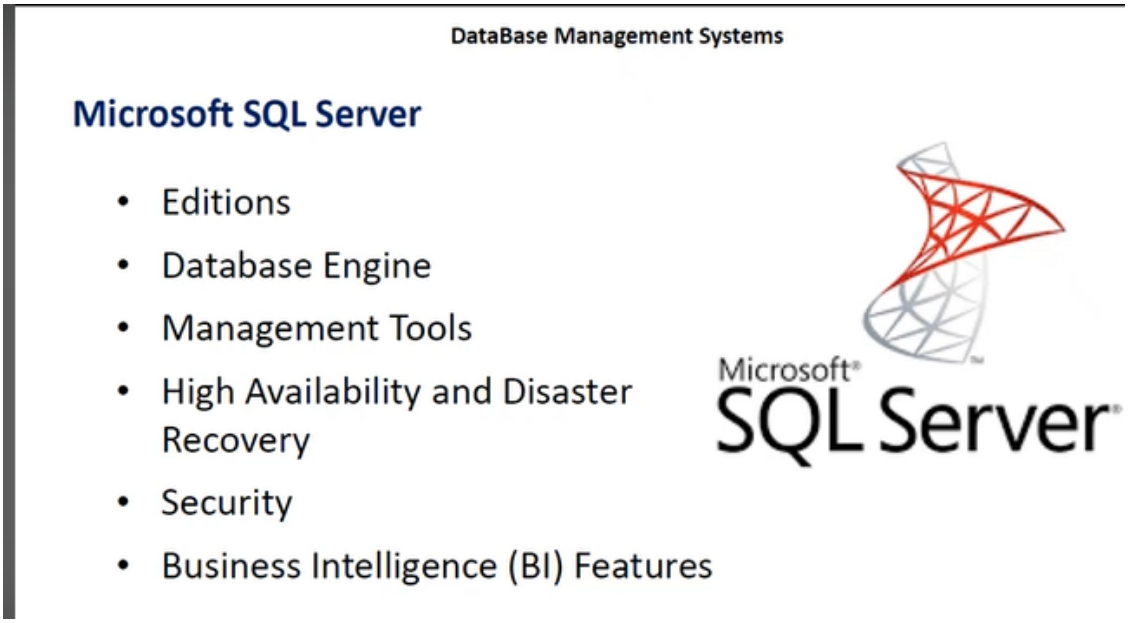
1.1 What Is a Database Management System (DBMS)?



Popular Database Management System software

A Database Management System (DBMS) is a software system used to store, organize, secure, update, and retrieve data whenever it is needed. Instead of every application storing data directly in its own separate files, a DBMS keeps all data in one structured, central location, allows multiple users to access it safely at the same time, and enforces the relationships between different pieces of data correctly. The image above lists some of the most widely used DBMS products today — Oracle Database, Microsoft SQL Server, MySQL, PostgreSQL, IBM Db2, SAP HANA, Amazon Aurora, Google Cloud Spanner, and MongoDB. Most of these (Oracle, SQL Server, MySQL, PostgreSQL, etc.) belong to the "Relational" DBMS (RDBMS) category, storing data as tables made of rows and columns. MongoDB, on the other hand, is a "NoSQL" DBMS, storing unstructured or semi-structured data in a JSON-like document format. In the rest of this book, we focus specifically on Microsoft SQL Server, which is one of the most widely used relational database systems in enterprise environments worldwide.

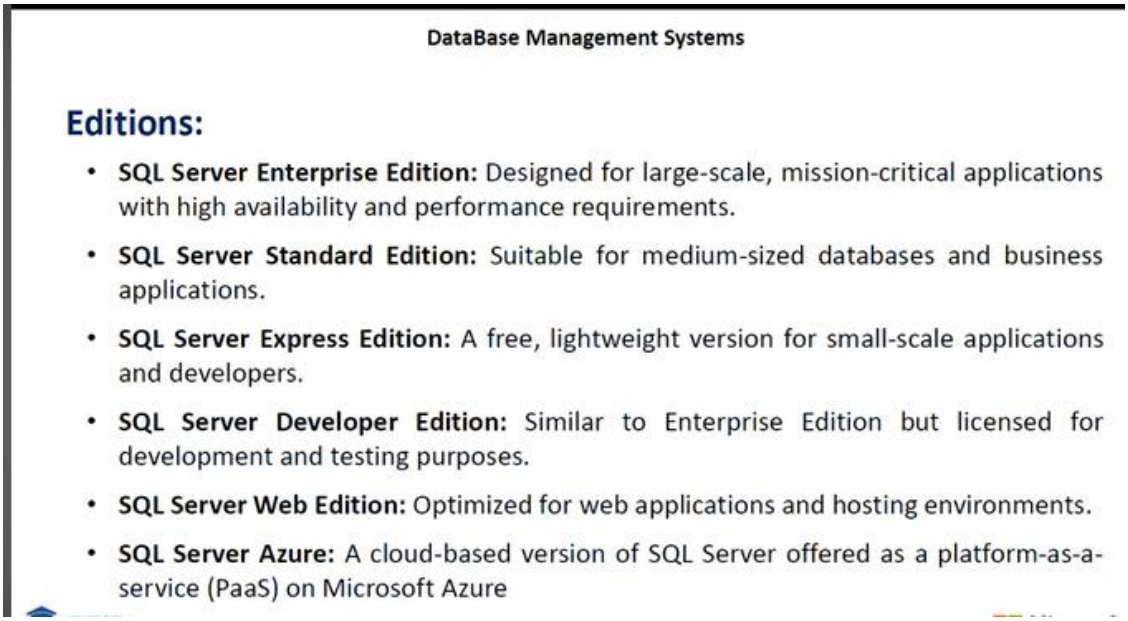
1.2 Microsoft SQL Server – Overall Overview



Key components of Microsoft SQL Server

Microsoft SQL Server is an enterprise-grade Relational Database Management System developed by Microsoft. This slide highlights its six main components — Editions (different versions built for different usage needs), Database Engine (the core that stores and processes data), Management Tools (software that helps administer SQL Server), High Availability and Disaster Recovery (mechanisms that keep the service running continuously), Security (data-protection features), and Business Intelligence (BI) Features (tools for analyzing data and building reports). Each of these components is explained separately in the sub-sections below.

1.3 SQL Server Editions



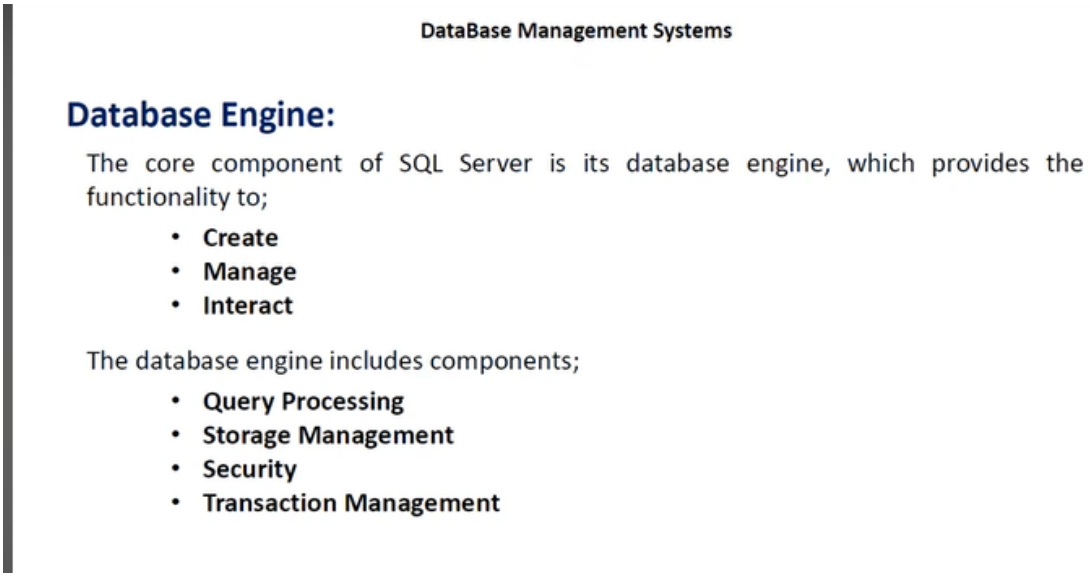
The different editions of SQL Server 2022

SQL Server is released in several editions to suit different usage needs and budgets:

- Enterprise Edition — designed for large-scale, mission-critical applications, meeting the highest High Availability and performance requirements. It requires the most expensive license and unlocks every available feature.
- Standard Edition — suitable for medium-sized databases and business applications; it lacks some of the advanced features found in Enterprise Edition (such as certain HA options) but costs less.
- Express Edition — a free edition for small-scale applications and developers, limited to a maximum database size of 10GB.
- Developer Edition — offers the same feature set as Enterprise Edition, but is free and licensed only for development and testing use, not for production.
- Web Edition — optimized for web hosting providers and web-facing applications, offered at a lower cost.
- SQL Server Azure — a cloud-based edition offered as a Platform-as-a-Service (PaaS) on Microsoft Azure; a database can be run in the cloud without purchasing any server hardware.

For the lab practical in this book, we use the "Evaluation" edition (a free 180-day trial with all the features of Enterprise Edition) — this is the most suitable choice for learning and practice.

1.4 Database Engine



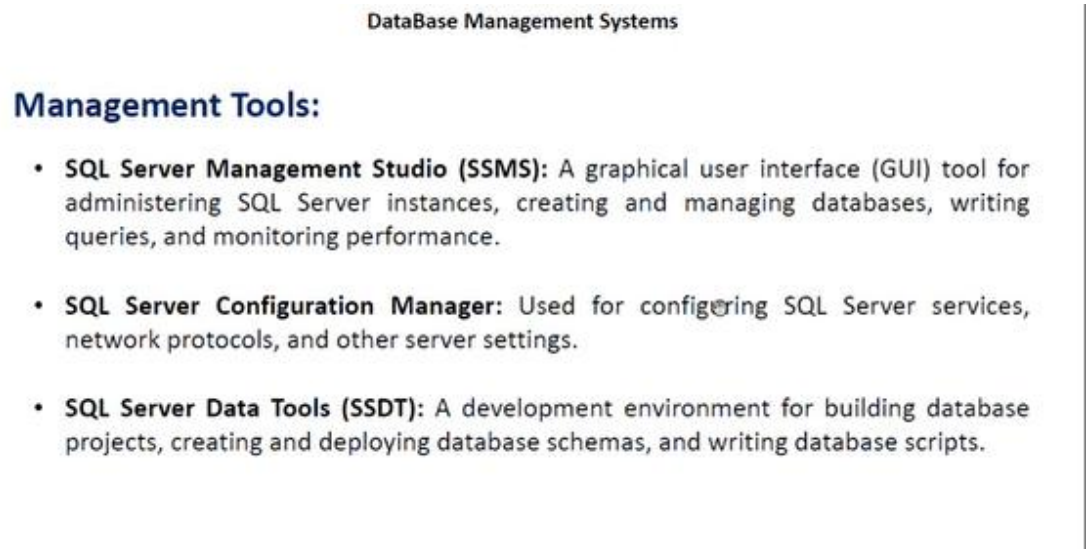
SQL Server Database Engine

The Database Engine is the heart (core component) of SQL Server. It handles three fundamental functions: creating a database, managing it, and interacting with it. Within the Database Engine, four main sub-components operate together:

- Query Processing — analyzes an SQL query you write, finds the most efficient way to execute it (by building an execution plan), and returns the result.

- Storage Management — physically organizes and manages data pages, files, and filegroups on disk.
- Security — controls login/user authentication and permissions (access control).
- Transaction Management — enforces the ACID properties (Atomicity, Consistency, Isolation, Durability) to maintain data accuracy when multiple users modify data at the same time.

1.5 Management Tools

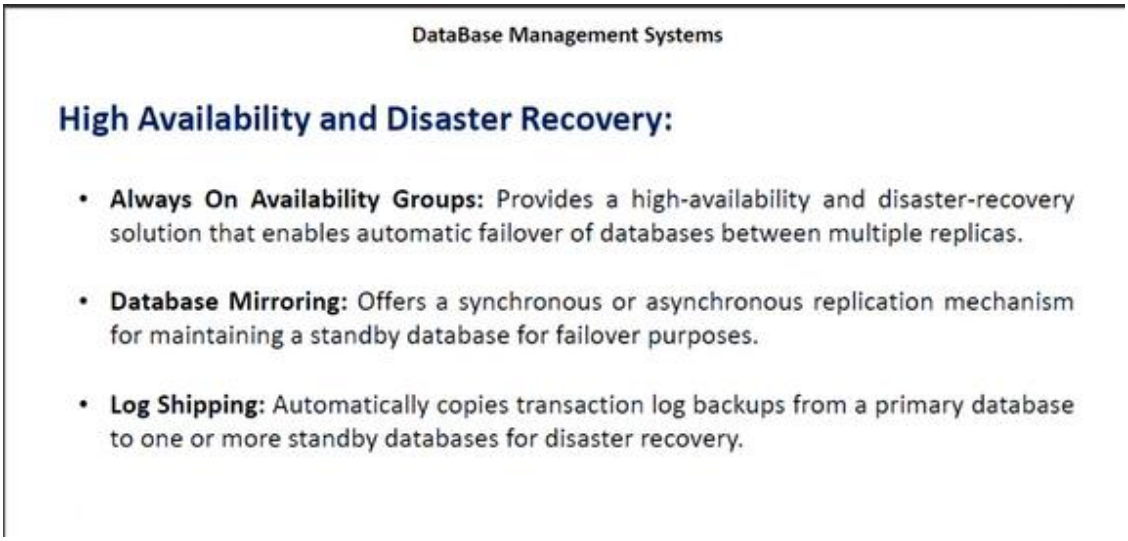


SQL Server Management Tools

Microsoft provides three main tools for administering SQL Server:

- SQL Server Management Studio (SSMS) — a graphical (GUI) tool used to administer SQL Server instances, create and manage databases, write queries, and monitor performance. This is the tool we use most throughout the lab practical in this book.
- SQL Server Configuration Manager — used to start/stop/restart SQL Server services, configure network protocols (such as TCP/IP), and manage other server settings.
- SQL Server Data Tools (SSDT) — a Visual Studio–based development environment for building database projects, deploying database schemas, and writing database scripts.

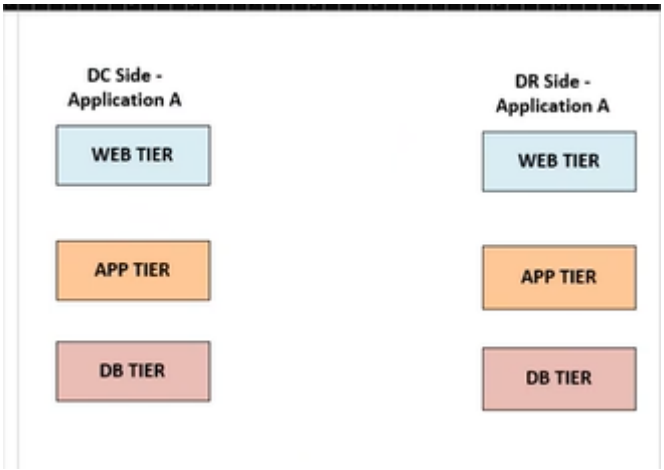
1.6 High Availability and Disaster Recovery (HA/DR)



High Availability and Disaster Recovery mechanisms

For business-critical systems, it is essential to keep the database server running without interruption, and if it does go down, to be able to recover it as quickly as possible. SQL Server provides three main mechanisms for this:

- Always On Availability Groups — a High Availability/Disaster Recovery solution that provides automatic failover of databases between multiple replicas (if one server fails, the workload automatically shifts to another server in the group).
- Database Mirroring — maintains a real-time copy of a primary database (either synchronously or asynchronously) on a standby server; if the primary server fails, the system can fail over to the standby server.
- Log Shipping — automatically copies transaction log backups from a primary database to one or more standby databases at set intervals — a simple and reliable method for Disaster Recovery.

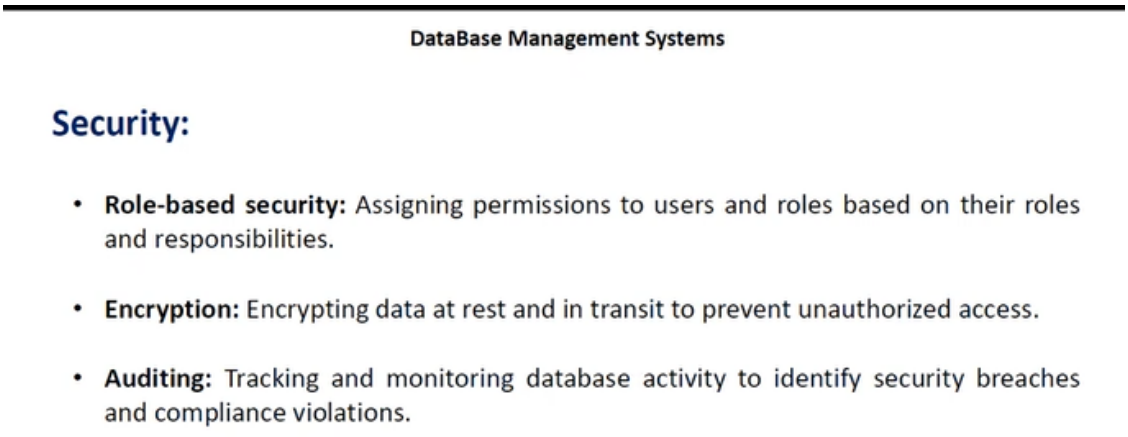


Application tier structure between the DC side and DR side

The diagram above shows how a real-world Disaster Recovery architecture should be planned. An application typically has three tiers — the Web Tier (which handles the user interface), the App Tier

(where business logic runs), and the DB Tier (the database engine). In a Disaster Recovery plan, all three of these tiers must exist not only on the DC (Data Center) side, but also in a matching, corresponding form on the DR (Disaster Recovery) side. That way, if a disaster (a fire, a power failure, a network outage) strikes the primary data center, traffic can be switched to the equivalent infrastructure on the DR side, allowing business operations to continue without interruption. This is one of the key aspects of Deployment Architecture planning, which we cover under "Preparation and Planning" in section 1.9.

1.7 Security



SQL Server security mechanisms

Protecting the data held in a database is one of the most important responsibilities of any DBMS. SQL Server offers three main security mechanisms:

- Role-based Security — instead of granting permissions directly to individual users, permissions are assigned to a "role" (e.g., SQLAdmins), and users are then added to or removed from that role to manage their access. (We will see this applied practically in the lab, when we create the SQLAdmins group.)
- Encryption — encrypting data both while it is stored on disk (at rest) and while it travels across the network (in transit), so it cannot be read by unauthorized parties.
- Auditing — continuously tracking and monitoring database activity (who accessed what data, when, and what changes were made) to detect security breaches and compliance violations.

1.8 Business Intelligence (BI) Features



SQL Server Business Intelligence tools

SQL Server does more than just store data — it also provides three Business Intelligence (BI) tools that help analyze data and support business decision-making:

- SQL Server Analysis Services (SSAS) — a multidimensional tool used to build OLAP (Online Analytical Processing) cubes and perform advanced data mining/analytics.
- SQL Server Integration Services (SSIS) — used to build and manage data integration workflows that extract data from various sources, transform/clean it, and load it into a destination database (ETL — Extract, Transform, Load).
- SQL Server Reporting Services (SSRS) — a reporting platform used to create and deliver interactive reports and dashboards. This is the component we attempt to install in Part 02 of the lab.

1.9 Preparation and Planning Before Installation

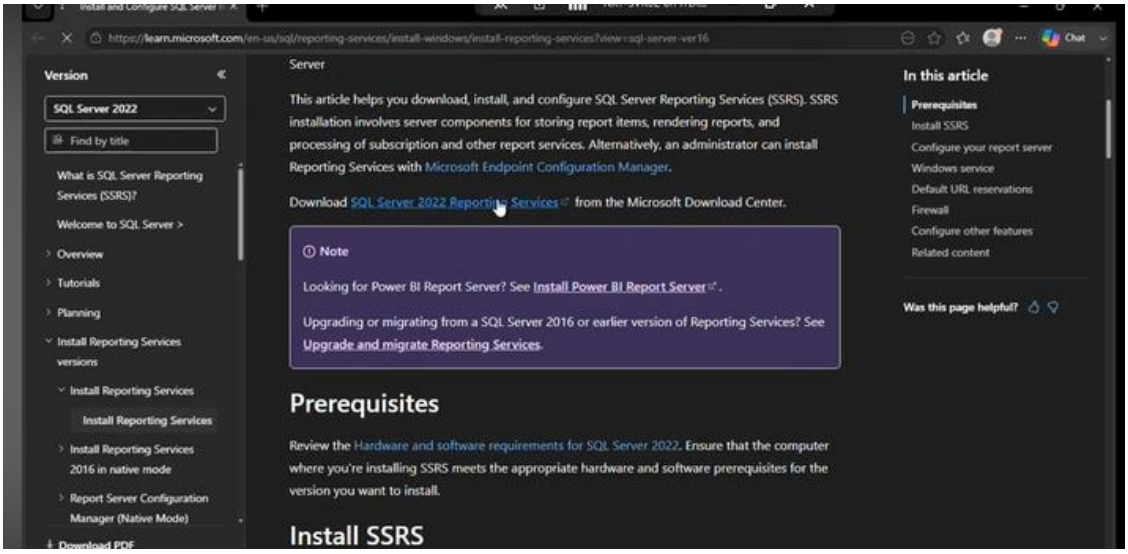


Points to plan for before installing SQL Server

Before installing SQL Server into a production environment, the following points should be fully planned out in advance. This list forms the basis for many of the practical decisions we make throughout the hands-on lab in Part 02:

- Define Requirements — determine the purpose of the database, the expected storage size, expected number of users, and other basic requirements.
- Hardware Sizing — plan whether the server's CPU, RAM, and disk space will be sufficient.
- Software Licensing — decide which edition is needed, and how many licenses are required.
- High Availability and Disaster Recovery — decide which of the mechanisms discussed in section 1.6 are needed.
- Security — plan the authentication mode and the permission structure.
- Backup and Recovery — put together a backup schedule and a recovery plan.
- Monitoring and Performance Tuning — plan a way to continuously observe the health of the server.
- Capacity Planning — plan resources with future growth in mind.
- Deployment Architecture — decide where the server should sit, and in which tier structure (see the diagram in section 1.6).
- Testing and Validation — after installation, verify that everything works correctly.

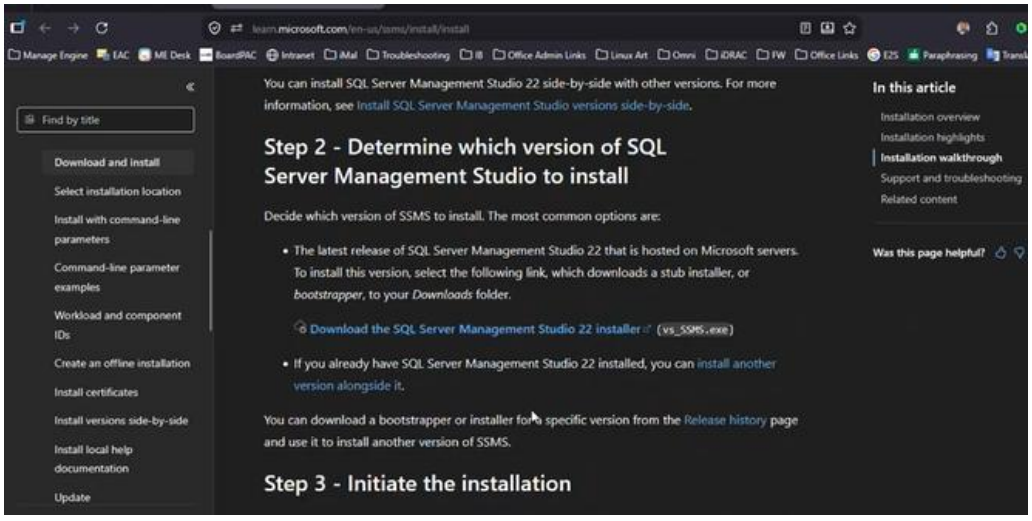
1.10 Reference — Microsoft Learn: Install SQL Server Reporting Services



Microsoft Learn – Install and Configure Report Server documentation

Before starting the lab, it is good practice to check Microsoft's official documentation (learn.microsoft.com) on how to install SQL Server Reporting Services (SSRS). It contains the prerequisites needed for SSRS installation, the download link, and step-by-step installation/configuration instructions. This same page is revisited in Part 02 when we attempt to install SSRS.

1.11 Reference — Microsoft Learn: Install SQL Server Management Studio



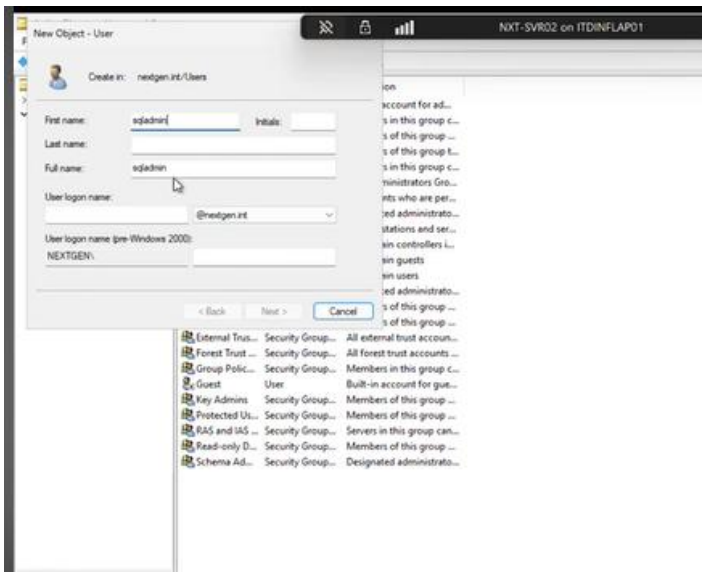
Microsoft Learn – Install SQL Server Management Studio documentation

Likewise, you can check the official documentation on how to install SQL Server Management Studio (SSMS). Under "Step 2 – Determine which version of SQL Server Management Studio to install," a link is provided to download the latest SSMS version. Since SSMS has not been bundled inside the main SQL Server installer since 2016, it must be downloaded and installed separately — this is shown practically in Part 02.

Part 02 – Installing SQL Server 2022: A Complete Hands-On Lab

In this part, using a Windows Server Active Directory domain (nextgen.int) and a Virtual Machine running on Hyper-V, we walk step by step through the entire process of installing SQL Server 2022, SQL Server Management Studio (SSMS), and SQL Server Reporting Services (SSRS), following each screenshot taken during the process. For every screenshot, we explain exactly what was done, why it was done, and how it connects to the steps before and after it.

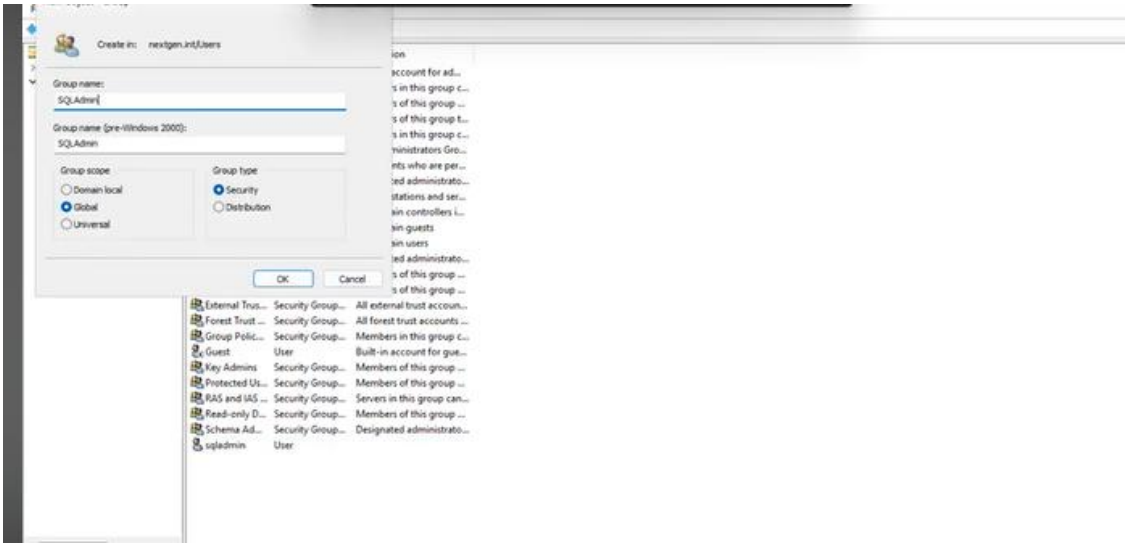
2.1 Creating a SQL Service Account in Active Directory



Active Directory Users and Computers – New Object: User

Rather than running a SQL Server service under a local Administrator account, it is good, widely accepted practice to create a dedicated Active Directory service account specifically for it. Here, the "Active Directory Users and Computers" (ADUC) tool is being used to create a new User object named "sqladmin," inside the Users container of the domain "nextgen.int," via New Object → User. Both the First name and Full name fields are set to "sqladmin." This account will later be used to run SQL Server services and to carry out database administration tasks.

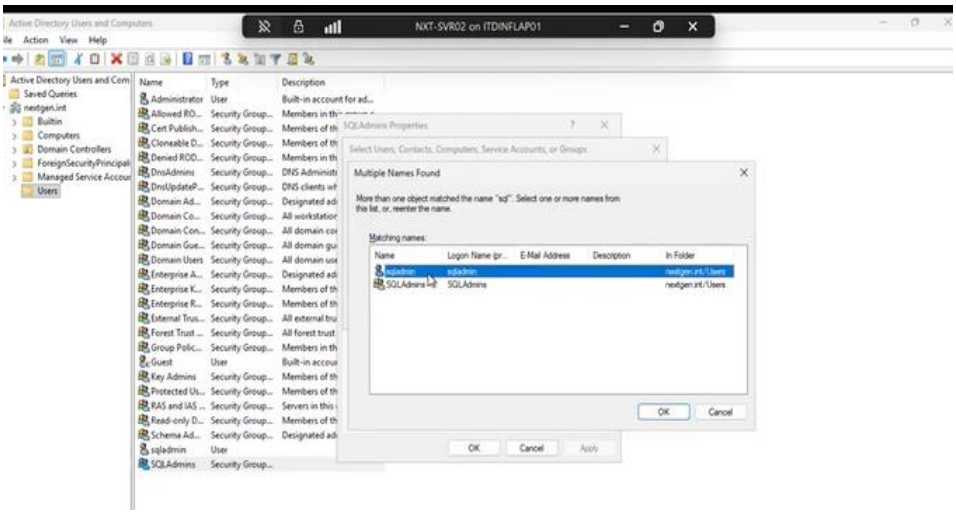
2.2 Creating a "SQLAdmins" Security Group



New Object: Group – SQLAdmin

Rather than granting permissions directly to an individual user account, it is a far more scalable approach to grant permissions to a group, and then add or remove the users who need that access (this is exactly the Role-based Security concept discussed in section 1.7). Here, a new security group named "SQLAdmin" is being created. The group scope is set to "Global" and the group type to "Security." (Security groups are used to grant permissions, while Distribution groups are used only for email distribution — not for permissions.) This group will later be granted sysadmin rights within SQL Server.

2.3 Adding the "sqladmin" User Account to the "SQLAdmins" Group

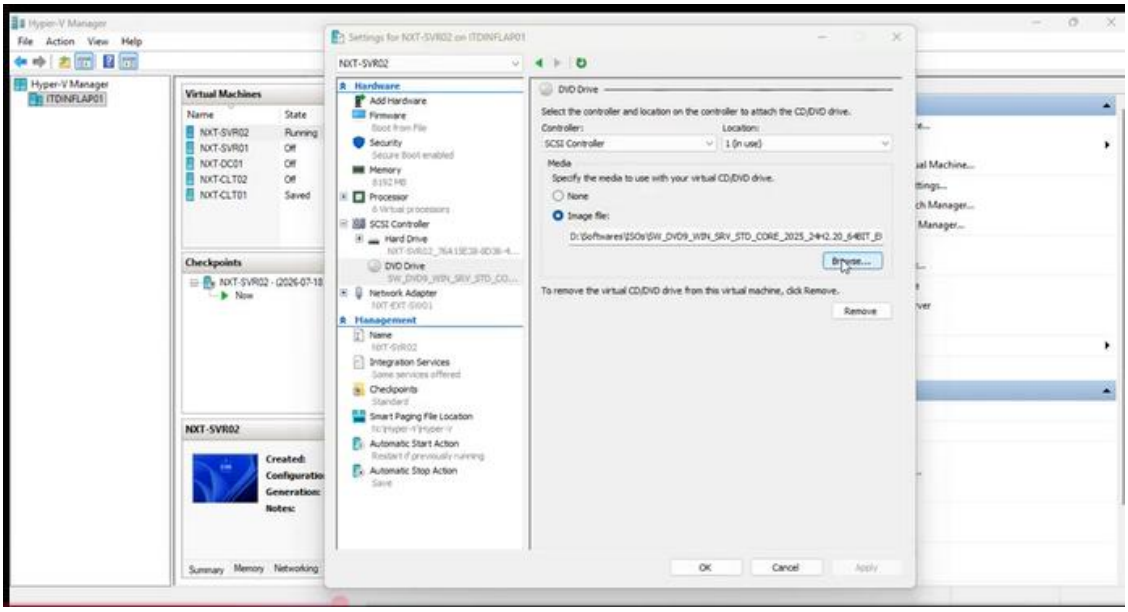


Select Users, Contacts, Computers, Service Accounts, or Groups

In this dialog box, searching for "sql" brings up a "Multiple Names Found" prompt — meaning two objects starting with "sql" (the user "sqladmin" and the group "SQLAdmins") were both matched. Here, the "sqladmin" user account created in section 2.1 is selected and added as a member of the "SQLAdmins" group created in section 2.2. After this step, during the SQL Server setup process,

sysadmin permission can be granted to this group as "NEXTGEN\SQLAdmins" (you will see this again in section 2.17).

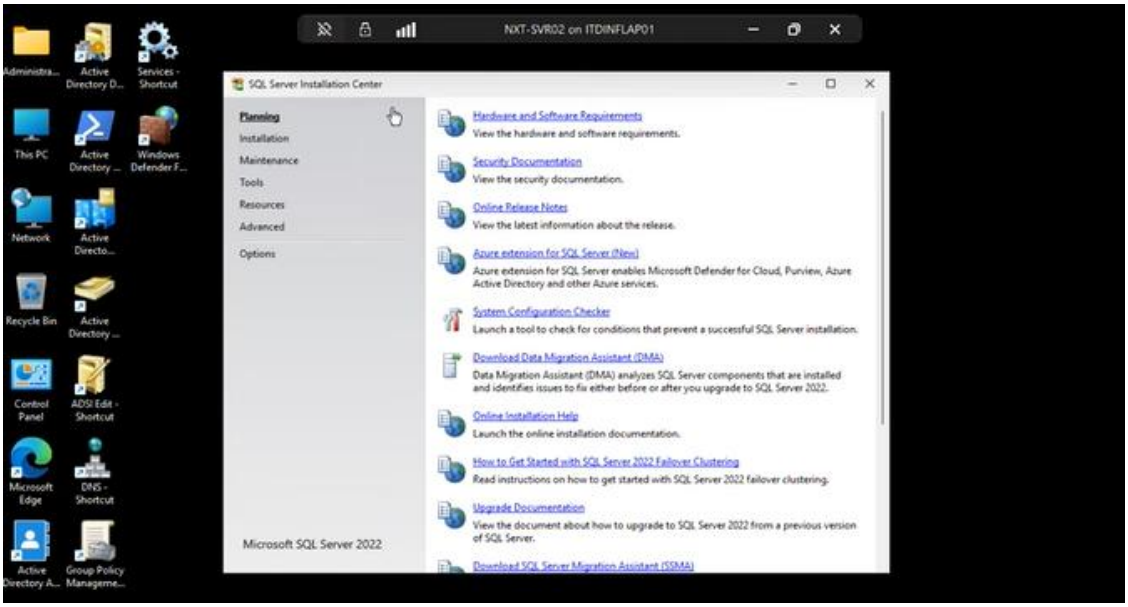
2.4 Attaching the SQL Server Installation Media (ISO) to the Virtual Machine



Hyper-V Manager – Settings – DVD Drive

Before installing SQL Server, the ISO image containing its installation files must be made available to the virtual machine on which SQL Server is going to be installed (NXT-SVR02). Just as you would insert a physical DVD into a drive on a physical server, on a Virtual Machine you go to Hyper-V Manager → Settings → SCSI Controller → DVD Drive → Media → Image file, browse to the path of the downloaded SQL Server 2022 ISO file, and "Attach" it. Once complete, from inside the virtual machine, this ISO appears exactly like a normal DVD drive, and the SQL Server setup can be launched from it.

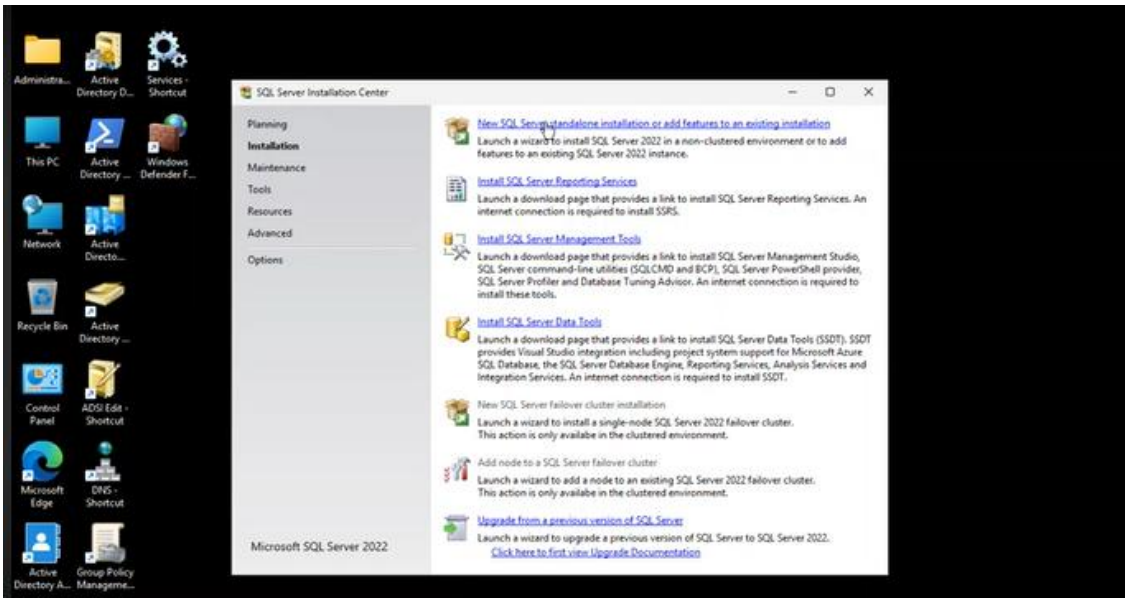
2.5 SQL Server Installation Center Opens — Planning Tab



SQL Server Installation Center – Planning

After attaching the ISO, opening it from inside the virtual machine automatically launches the "SQL Server Installation Center" (or you can manually run the setup.exe file found on the DVD drive). By default it opens on the "Planning" tab — this contains a range of links you can review before installing, such as Hardware and Software Requirements, Security Documentation, and the System Configuration Checker. These are not mandatory to click through, but reviewing them is recommended in a production environment.

2.6 Installation Tab — Starting a New Installation

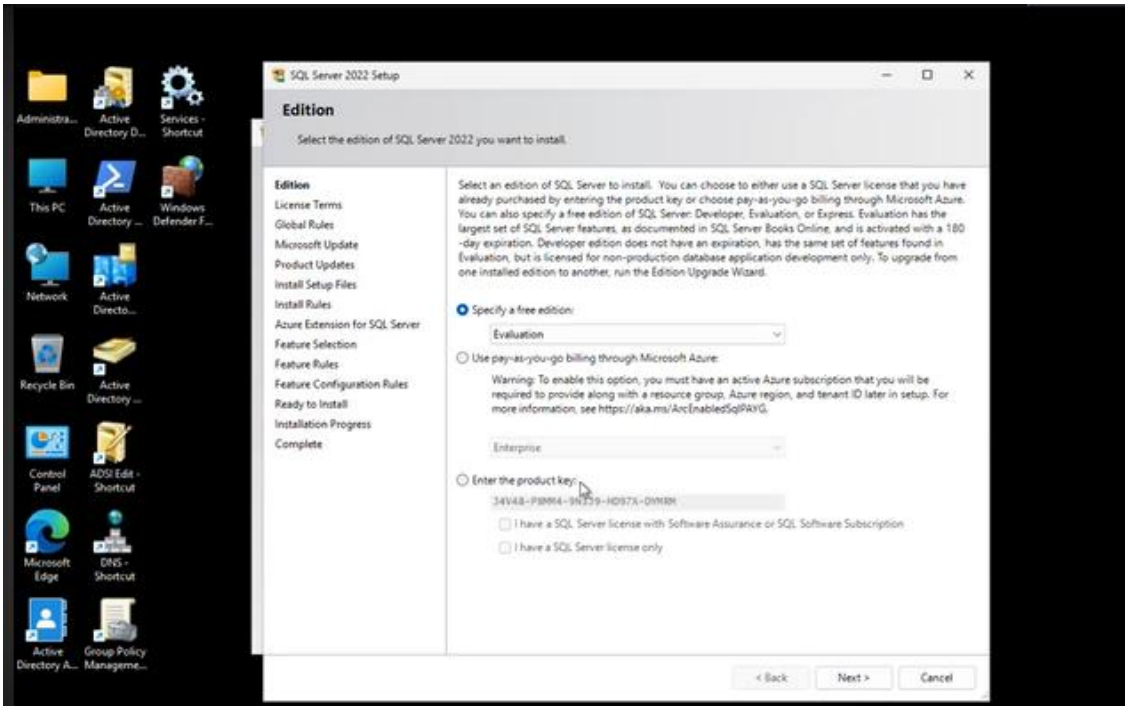


SQL Server Installation Center – Installation Tab

Switching to the "Installation" tab on the left shows several installation options. Selecting the first link, "New SQL Server stand-alone installation or add features to an existing installation," starts the wizard

for installing SQL Server 2022 fresh onto a non-clustered (single-server) environment. (The Failover Cluster options further down are only needed in large enterprise environments where multiple servers are clustered together.)

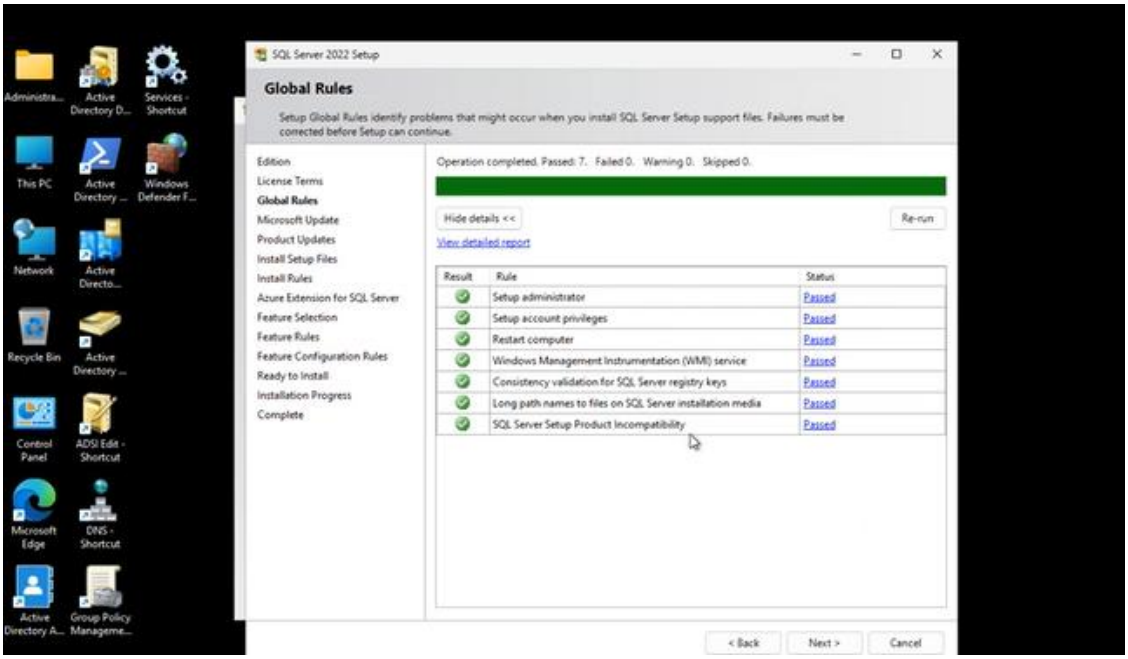
2.7 Selecting the Edition



SQL Server 2022 Setup – Edition

On this screen you select which SQL Server 2022 edition to install. Three options are available — a free edition (Evaluation, Express, or Developer), an edition billed pay-as-you-go through Azure, or entering an already-purchased product key. In this lab, "Evaluation" has been selected (a free 180-day trial including all Enterprise Edition features) — this is the most suitable choice for learning and testing purposes. In a production environment, an appropriately purchased license key must be entered.

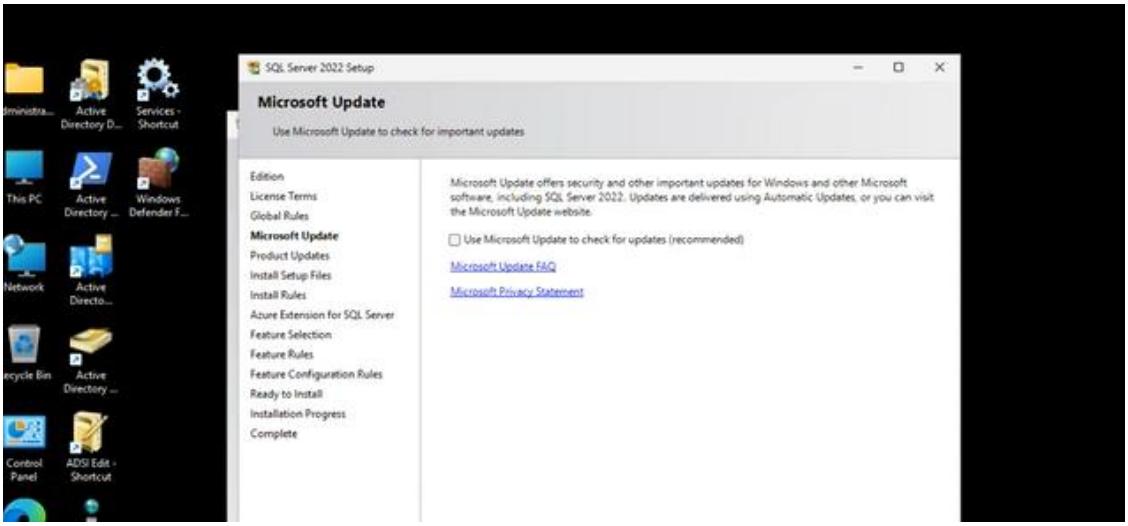
2.8 Global Rules Check



SQL Server 2022 Setup – Global Rules

The setup wizard automatically checks the current state of the server — whether Setup Administrator privileges exist, whether a computer restart is pending, whether the Windows Management Instrumentation (WMI) service is running, and 7 such basic checks in total, all showing "Passed." If any of these rules fail, they must be fixed before you can proceed. Once all checks pass, click "Next."

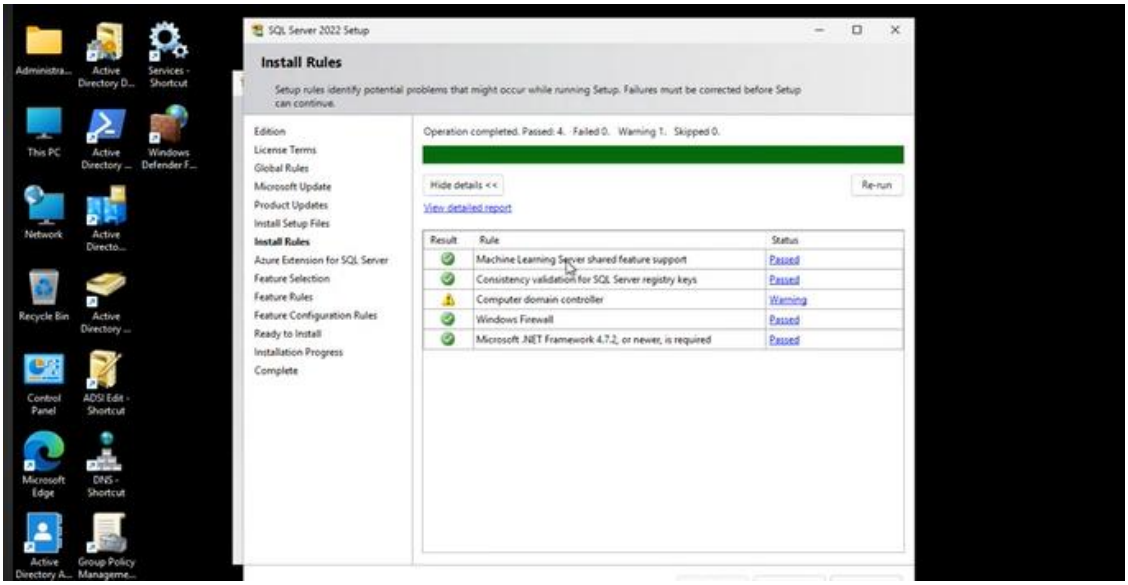
2.9 Microsoft Update



SQL Server 2022 Setup – Microsoft Update

This screen asks whether you want updates checked through Microsoft Update during installation itself. In this lab, the box has been left unchecked (i.e., updates are not checked at install time) before proceeding — this is a common practice in isolated or tightly controlled patch-management environments, where updates are applied separately, later, in a controlled manner.

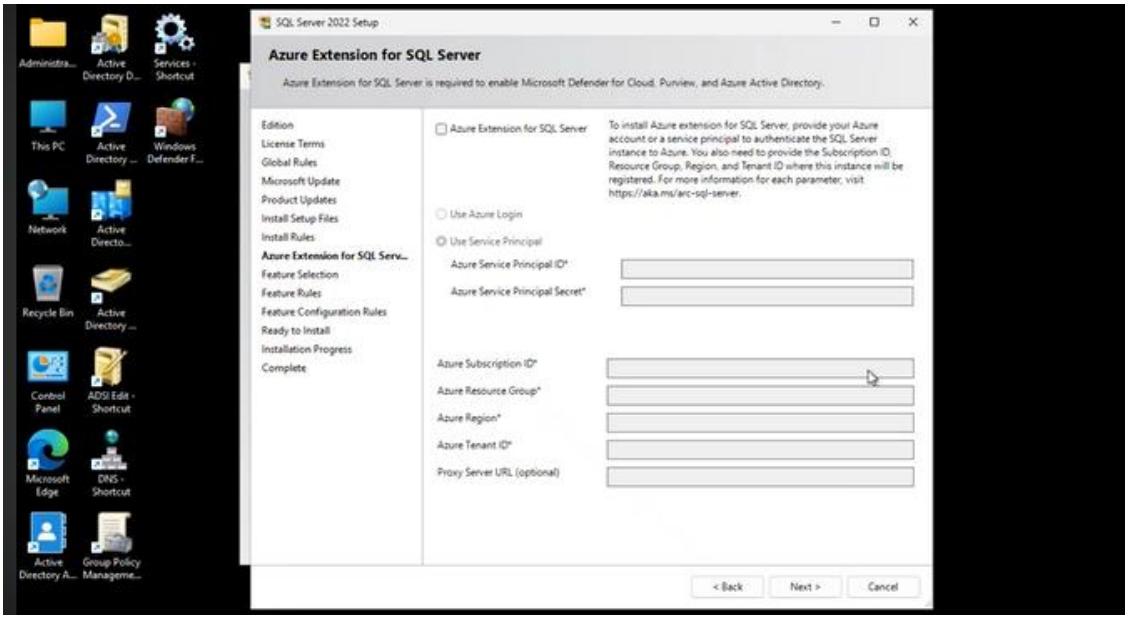
2.10 Install Rules Check — Domain Controller Warning



SQL Server 2022 Setup – Install Rules

A further set of rules (5) is checked here. Four of them show "Passed," but the "Computer domain controller" rule shows a "Warning." This is because the target server on which SQL Server is being installed is also functioning as an Active Directory Domain Controller. For security and performance reasons, Microsoft does not recommend installing SQL Server (and particularly companion services such as Reporting Services) on a Domain Controller. However, since this is only a "Warning" and not a "Failed" result, installation can proceed without being blocked. This warning becomes a real problem later on (in section 2.33) when we attempt to install SSRS — so it is worth paying attention to this message now.

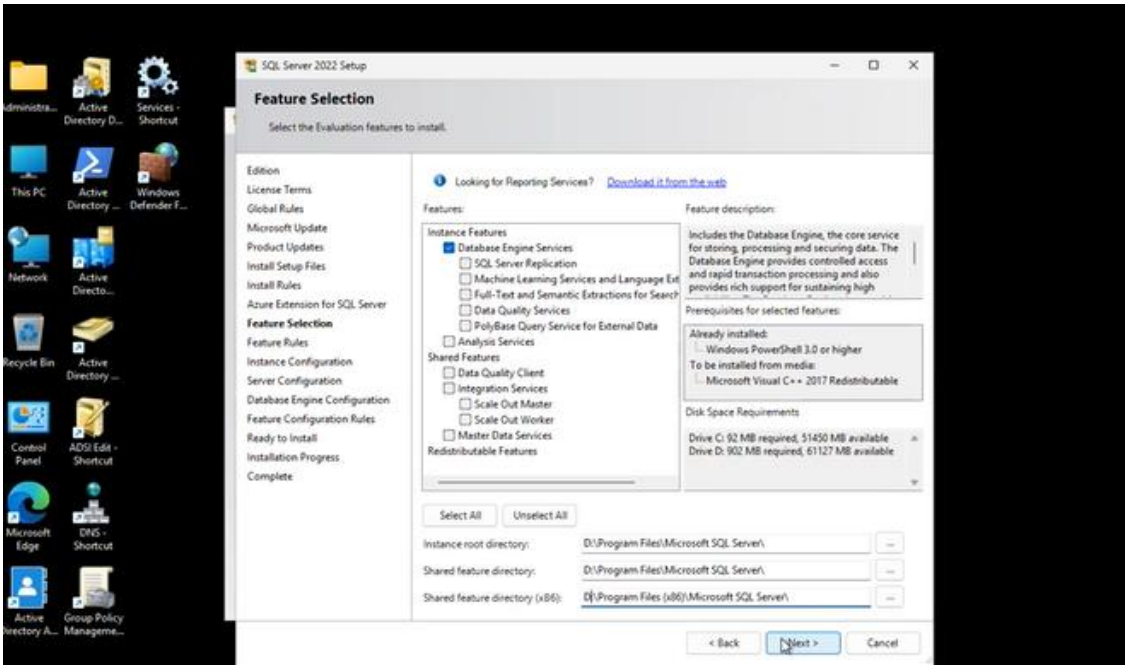
2.11 Azure Extension for SQL Server



SQL Server 2022 Setup – Azure Extension for SQL Server

A new feature in SQL Server 2022, the "Azure Extension for SQL Server," allows the SQL Server instance to be registered with Azure cloud services such as Microsoft Defender for Cloud, Microsoft Purview, and Azure Active Directory. Since this lab does not connect to Azure, this option is left unchecked (skipped) before proceeding.

2.12 Feature Selection — Selecting Database Engine Services

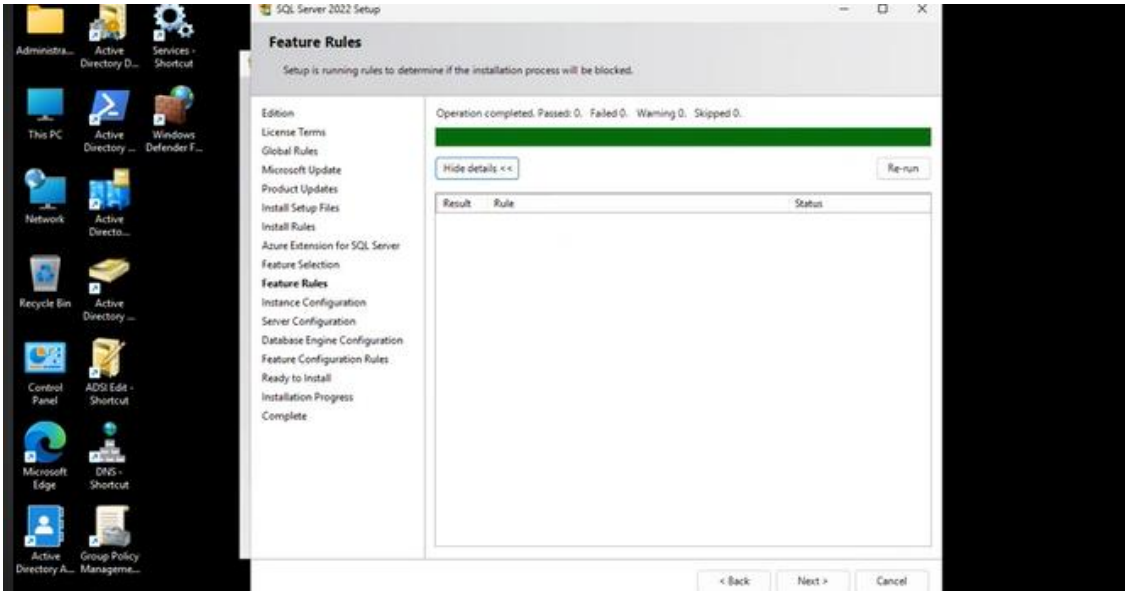


SQL Server 2022 Setup – Feature Selection

This screen selects which SQL Server features to install. For this lab, only the core feature needed — "Database Engine Services" (along with SQL Server Replication) — has been checked; this is the core component responsible for storing, processing, and securely holding data. Also, both the "Instance root directory" and "Shared feature directory" paths have been changed from the default C: drive to the D: drive.

> *Important note — why install on a separate partition?: It is good, widely accepted practice to keep SQL Server's program files and data files off the C: drive (where the Windows Operating System is installed) and instead place them on a separate partition (here, the D: drive). There are several reasons for this: (1) it prevents the C: drive from filling up and leaving the operating system without space, (2) placing database files on a separate disk can improve performance, (3) it makes backup/restore and disaster recovery tasks easier, and (4) it reduces the risk of data loss during a future SQL Server upgrade or reinstall.*

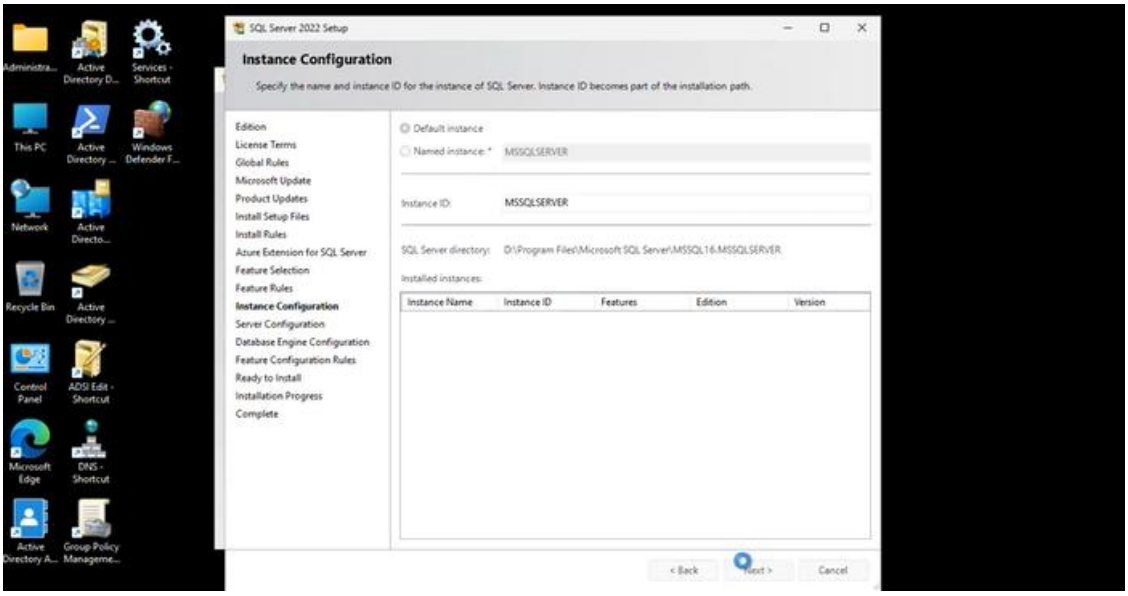
2.13 Feature Rules Check



SQL Server 2022 Setup – Feature Rules

The setup wizard automatically re-checks whether any prerequisites are missing for the selected features (here, only Database Engine Services). Since no rules were applicable in this case (0 passed, 0 failed), the list is shown empty, and the wizard automatically proceeds to the next step.

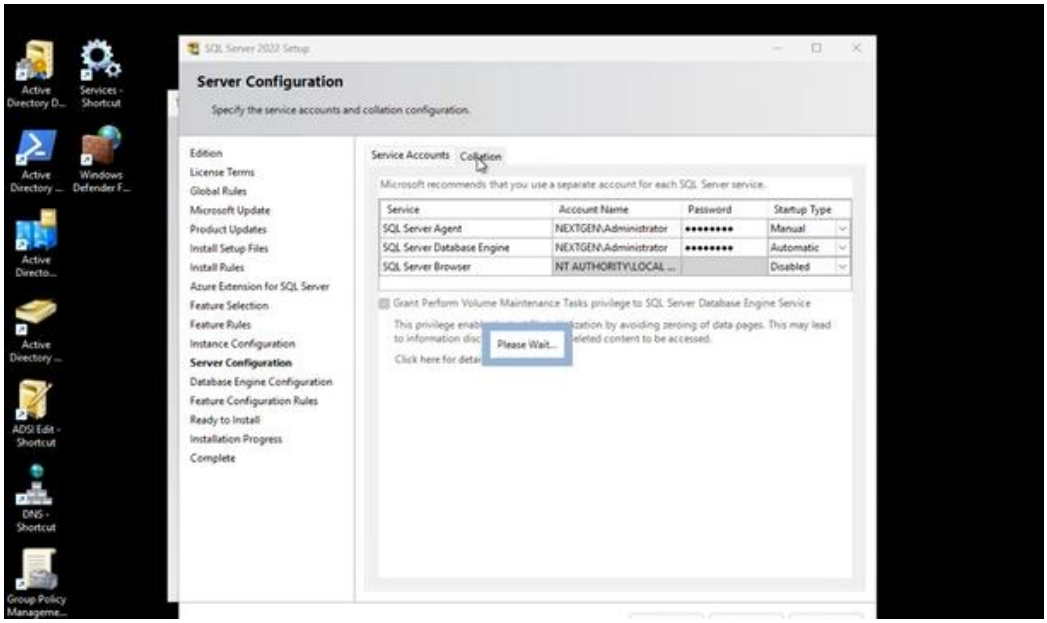
2.14 Instance Configuration



SQL Server 2022 Setup – Instance Configuration

A single physical or virtual server can host multiple SQL Server instances. Choosing "Default instance" means this SQL Server can be connected to using just the server name (without needing an instance name suffix) — the Instance ID is automatically set to "MSSQLSERVER." The SQL Server directory shown here points to the D: drive, in line with the decision made in section 2.12. (If multiple instances are needed, the "Named instance" option can be used to install more than one.)

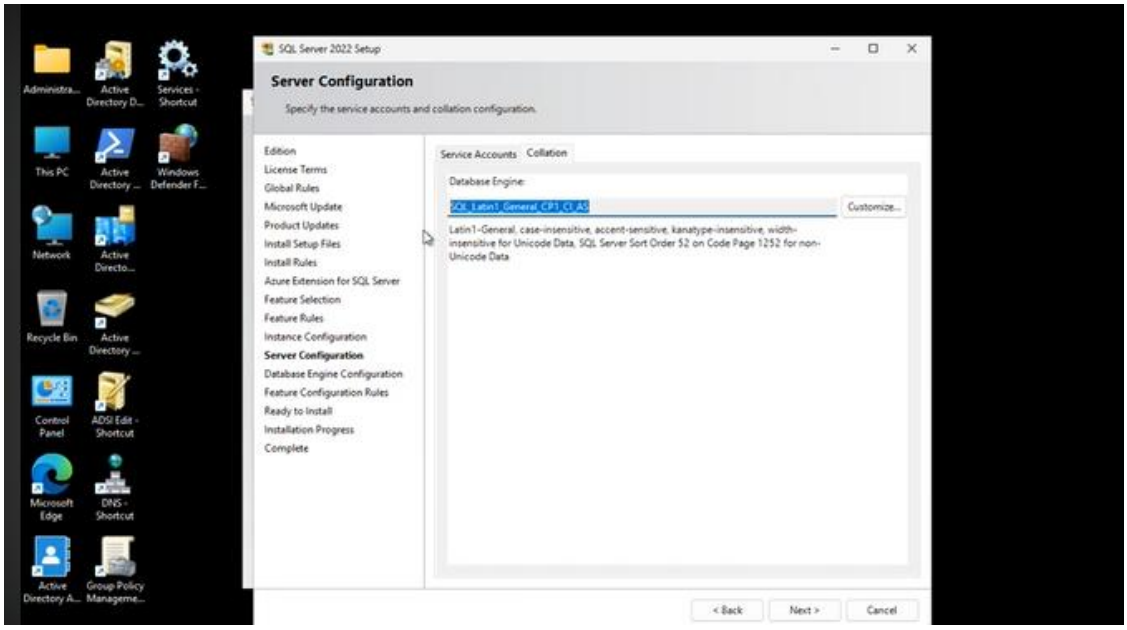
2.15 Server Configuration — Service Accounts



SQL Server 2022 Setup – Server Configuration – Service Accounts

Each SQL Server service (SQL Server Agent, SQL Server Database Engine, SQL Server Browser) must run under a Windows account. Here, both "SQL Server Agent" (the service that runs scheduled jobs and automation tasks) and "SQL Server Database Engine" (the core engine) have been configured to run under the domain account "NEXTGEN\Administrator." "SQL Server Browser" (a service that helps client applications discover named instances) is left under the built-in "NT AUTHORITY\LOCAL SERVICE" account, with its Startup Type kept as "Disabled" (since this lab only uses a default instance, the Browser service is not mandatory). In a production environment, for security reasons, it is better to use a dedicated, low-privilege service account — such as the "sqladmin" account created in section 2.1 — rather than the Administrator account.

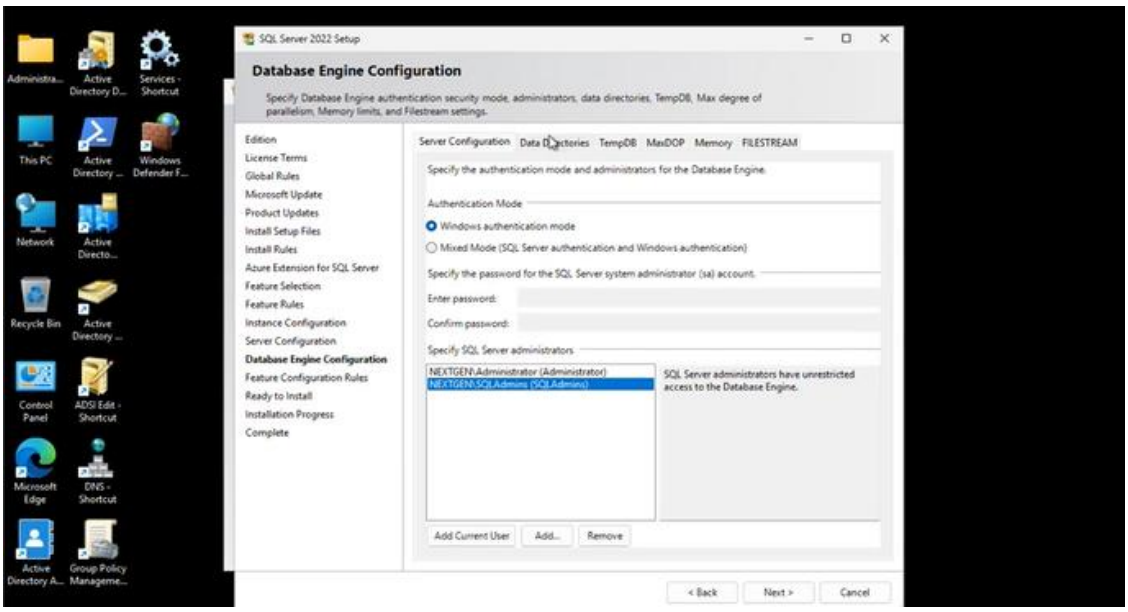
2.16 Server Configuration — Collation



SQL Server 2022 Setup – Server Configuration – Collation

The Collation setting determines how the Database Engine sorts and compares text data. The default, "SQL_Latin1_General_CP1_CI_AS," is: a Latin1-General character set, Case-Insensitive ("CI" — treats "A" and "a" as equal), and Accent-Sensitive ("AS" — treats "a" and "á" as different). This is a suitable default collation for English/general use. It is worth noting that changing the collation after a database has been created is a difficult and complex task, so it is important to choose the correct collation upfront, based on your organization's language/locale requirements.

2.17 Database Engine Configuration — Authentication Mode



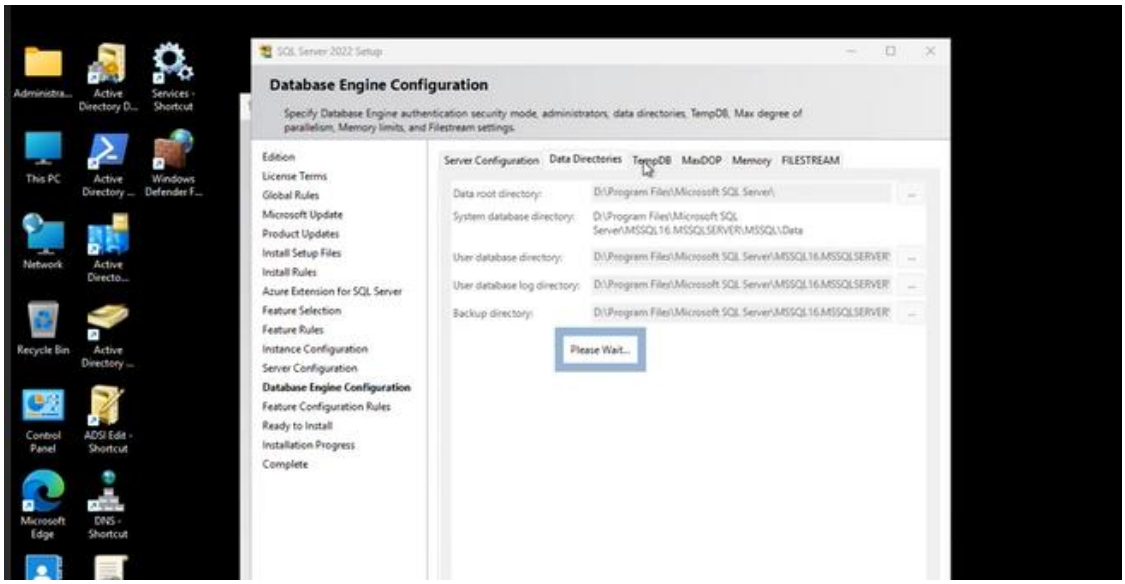
SQL Server 2022 Setup – Database Engine Configuration – Server Configuration

This is an extremely important security setting. Two options are available:

- Windows Authentication Mode — only accounts with a Windows/Active Directory account can connect to SQL Server (this is the option selected in this lab — it is more secure, since there is no separate password to manage).
- Mixed Mode (SQL Server authentication and Windows authentication) — in addition to the above, you can also connect using a username/password stored within SQL Server itself — required for applications that are not joined to the domain.

Below that, under "Specify SQL Server administrators," accounts granted unrestricted (sysadmin) access to this SQL Server instance are added — "NEXTGEN\Administrator" is included, along with the "NEXTGEN\SQLAdmins" group created in sections 2.2/2.3. This means that any user added to the SQLAdmins group in the future will automatically receive full admin access without needing to reconfigure SQL Server itself — this is the practical benefit of role-based security.

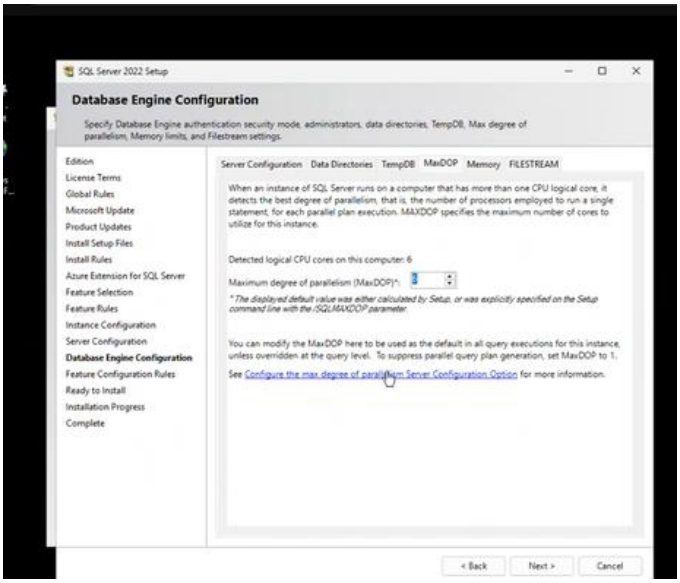
2.18 Database Engine Configuration — Data Directories



SQL Server 2022 Setup – Database Engine Configuration – Data Directories

On this tab, you can customize the folder paths where SQL Server stores various types of files separately — the Data root directory, System database directory, User database directory, User database log directory, and Backup directory. Here, all of these have been pointed to the D: drive, consistent with the decision made in section 2.12. In large-scale production environments, it is recommended to place these directories on separate physical disks, both to improve performance (by allowing data files and log files to be written to different disks in parallel) and to make backup/recovery tasks easier.

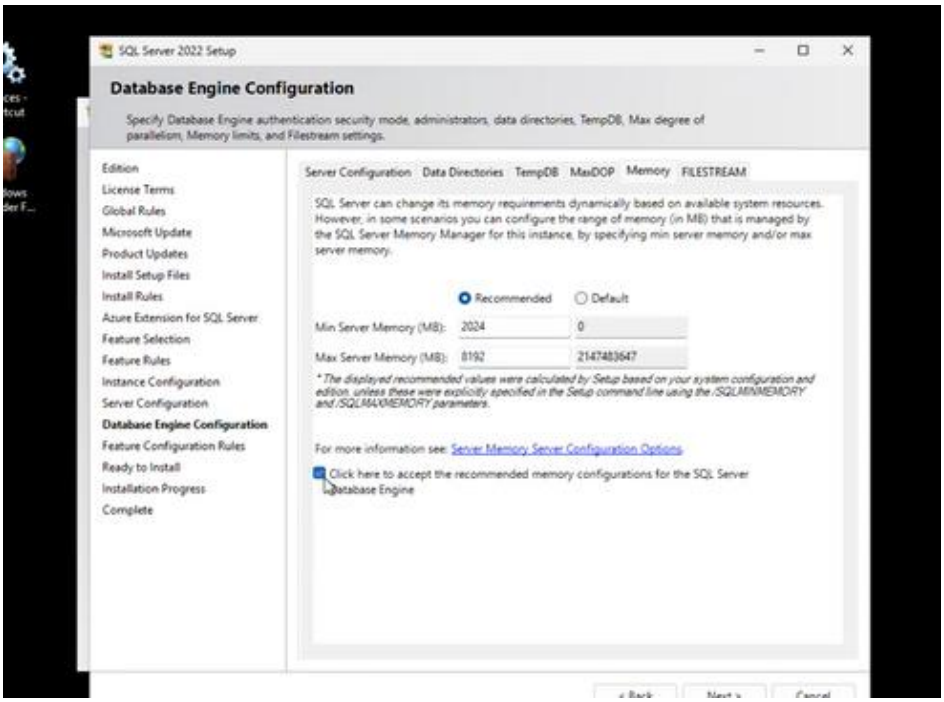
2.19 Database Engine Configuration — MaxDOP



SQL Server 2022 Setup – Database Engine Configuration – MaxDOP

"Maximum Degree of Parallelism" (MaxDOP) limits how many CPU cores SQL Server can use simultaneously to execute a single query. This server has 6 logical CPU cores detected, and the setup wizard has automatically suggested a recommended value (here, 3). This value is used by default for any query going forward, unless it is overridden at the individual query level. Tuning MaxDOP correctly has a significant impact on performance, whether the workload is Online Transaction Processing (OLTP — many small, fast queries) or a Data Warehouse (large, complex queries).

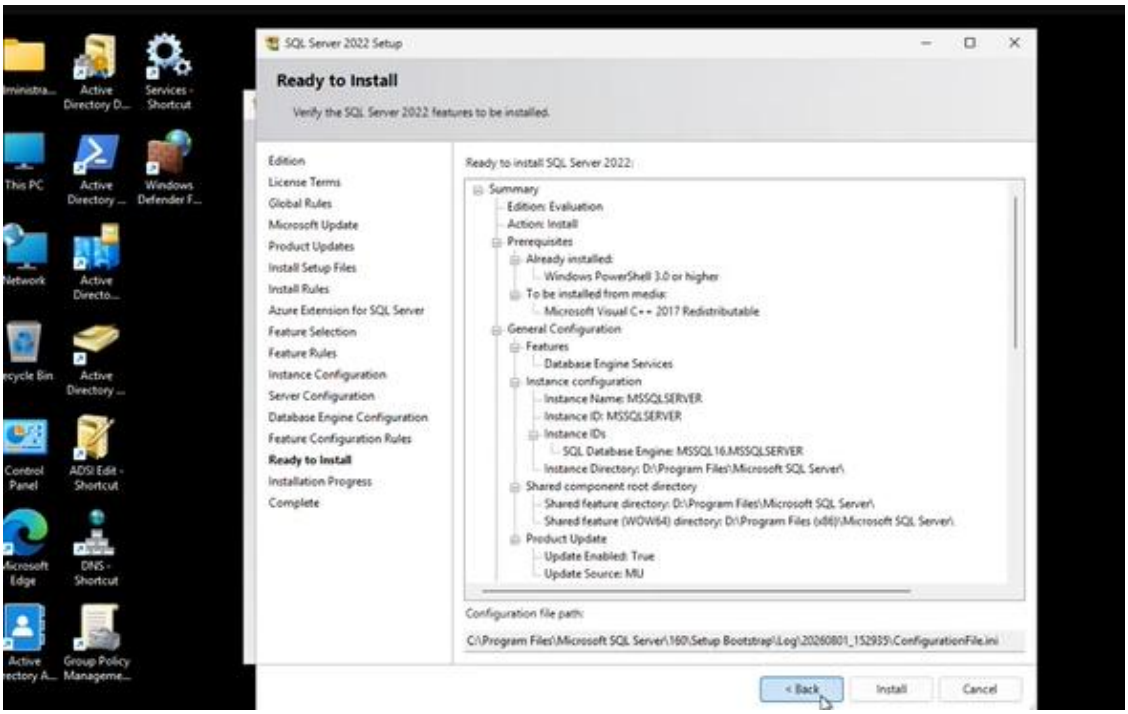
2.20 Database Engine Configuration — Memory



SQL Server 2022 Setup – Database Engine Configuration – Memory

This tab configures the minimum and maximum amount of RAM SQL Server is allowed to use. The "Recommended" option is selected, and based on the server's total RAM, the Min Server Memory has been automatically calculated as 2024 MB and the Max Server Memory as 8192 MB (under the "Default" option, there would be no cap at all on max memory — meaning SQL Server could consume all of the server's available RAM). Capping the maximum memory is especially important here because this server is also acting as an Active Directory Domain Controller, and this setting prevents SQL Server from consuming all the RAM and starving the Domain Controller services of the memory they need.

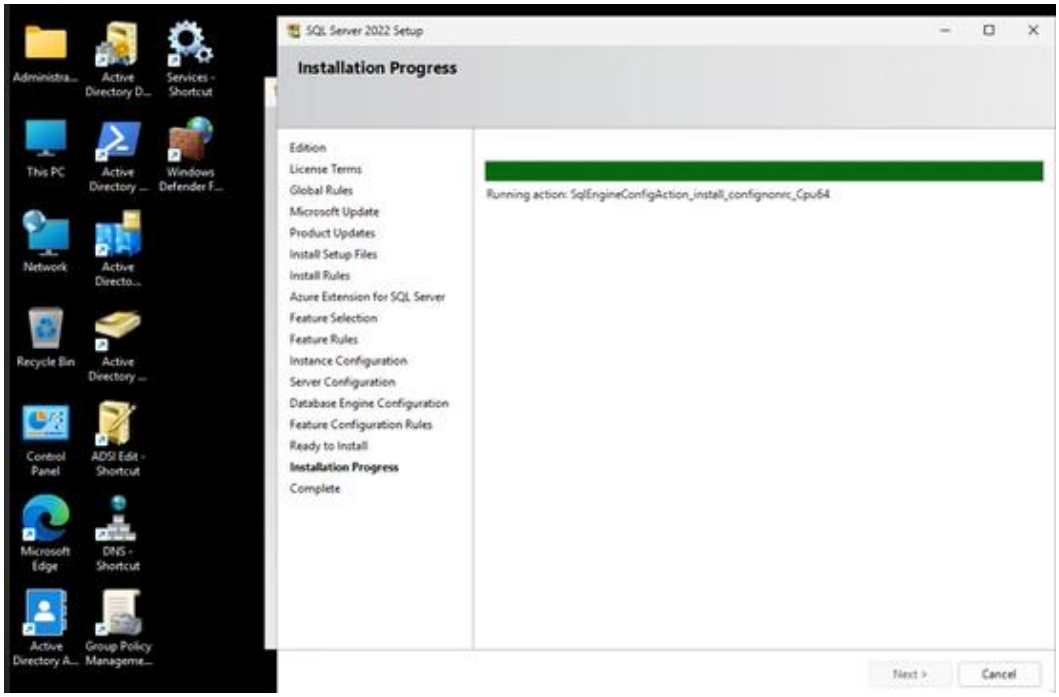
2.21 Ready to Install — Final Review



SQL Server 2022 Setup – Ready to Install

Before the actual installation begins, this screen shows a summary of everything selected — Edition, Features, Instance Configuration, Directory Paths, Product Update Settings, and more. Reviewing this summary carefully before clicking Install allows you to click "Back" and correct anything that's wrong — some settings become difficult to change once the installation process has started.

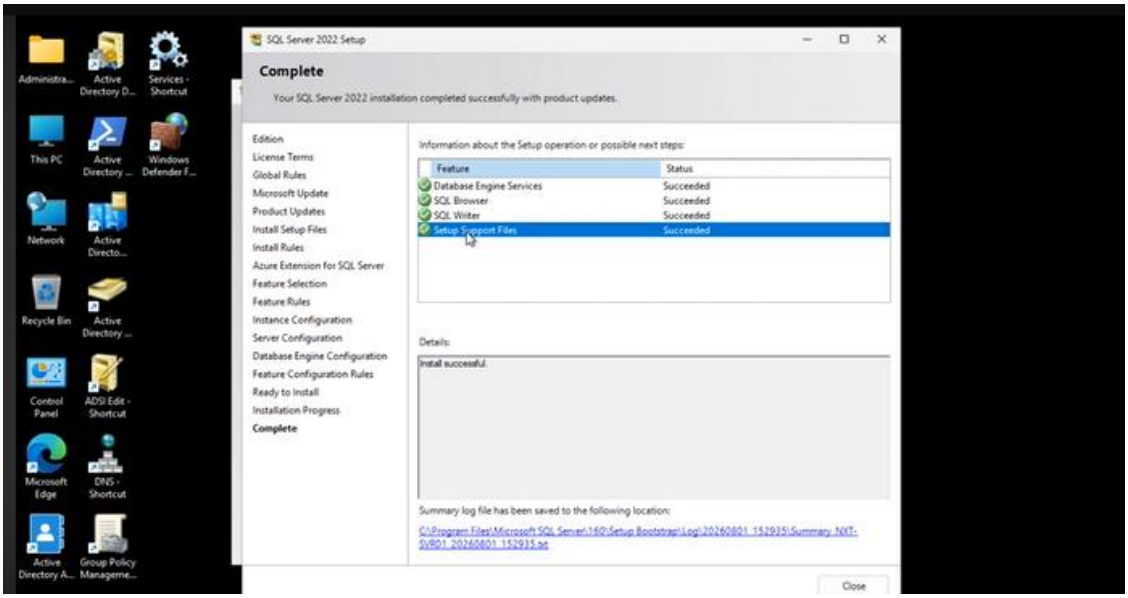
2.22 Installation Progress



SQL Server 2022 Setup – Installation Progress

After clicking "Install," the actual installation process begins. A progress bar and the label "Running action:" show the specific step currently in progress (here, "SqlEngineConfigAction_install_configrc_Cpu64"). This process can take several minutes depending on the server's hardware capacity — the window should not be closed while it is running.

2.23 Installation Complete



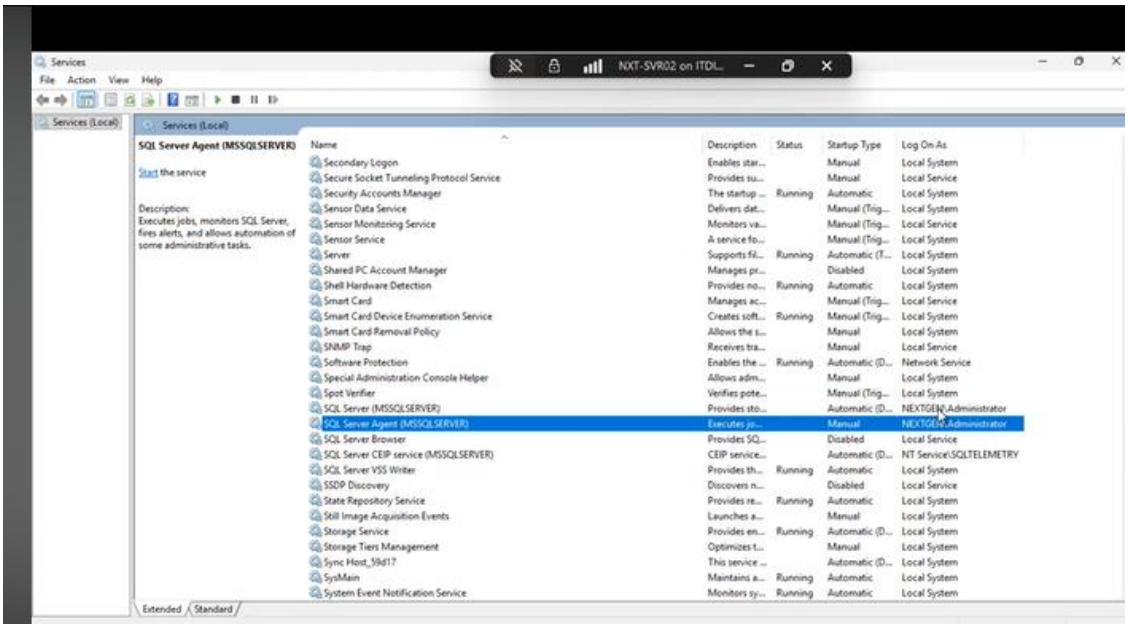
SQL Server 2022 Setup – Complete

Once installation completes successfully, the status of each installed feature is shown — "Database Engine Services," "SQL Browser," "SQL Writer," and "Setup Support Files" — all showing

"Succeeded" confirms that the installation finished without any errors. Below that, the location of the Summary Log File is provided, which can be referred to for troubleshooting if needed.

After installation completes, it is recommended to restart the server to ensure SQL Server services start up correctly. Restarting after installing major software ensures that file locks and registry changes are fully applied, and confirms that all services start up cleanly.

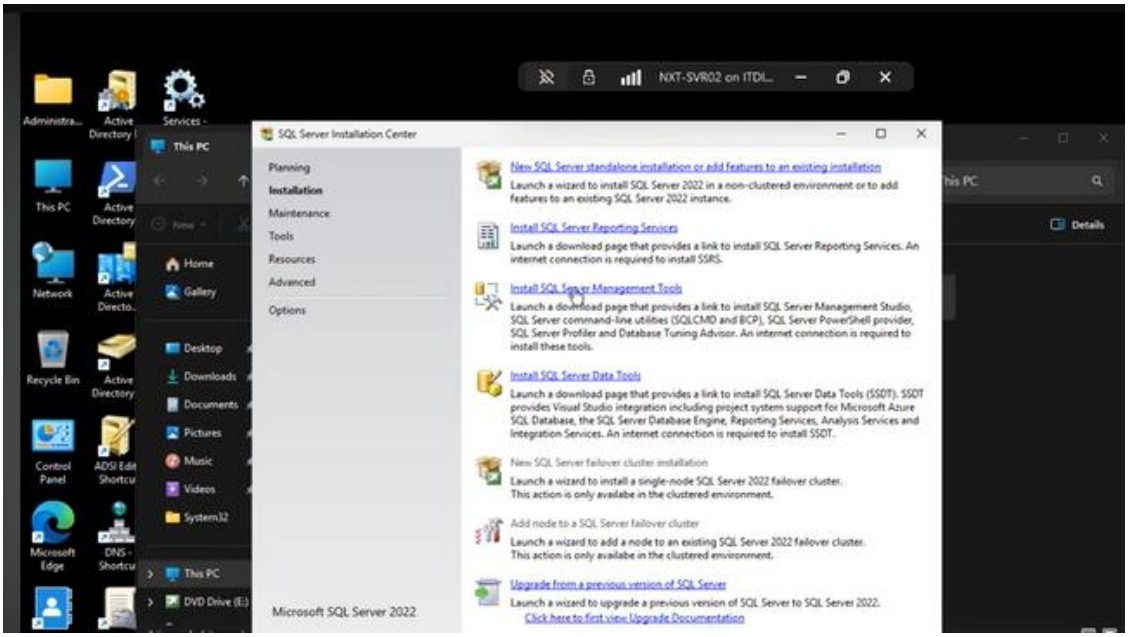
2.24 Verifying Services After Restart



Windows Services Console (services.msc)

After restarting the server, open the Windows "Services" console (services.msc) to check that the SQL Server services are running correctly. Here you can see both "SQL Server (MSSQLSERVER)" and "SQL Server Agent (MSSQLSERVER)" listed, with the "Log On As" column showing "NEXTGEN\Administrator" (as configured in section 2.15), and the Status showing "Running" — confirming that the installation was fully successful and the SQL Server engine is now ready to accept client connections.

2.25 Beginning the SQL Server Management Studio (SSMS) Installation



SQL Server Installation Center – Install SQL Server Management Tools

Even though the Database Engine has been installed, SQL Server Management Studio (SSMS) must be installed separately in order to manage it graphically (SSMS has not been bundled inside the main installer since SQL Server 2016). Returning once again to the SQL Server Installation Center's Installation tab, this time the "Install SQL Server Management Tools" link is clicked — this redirects to the SSMS download page in the browser.

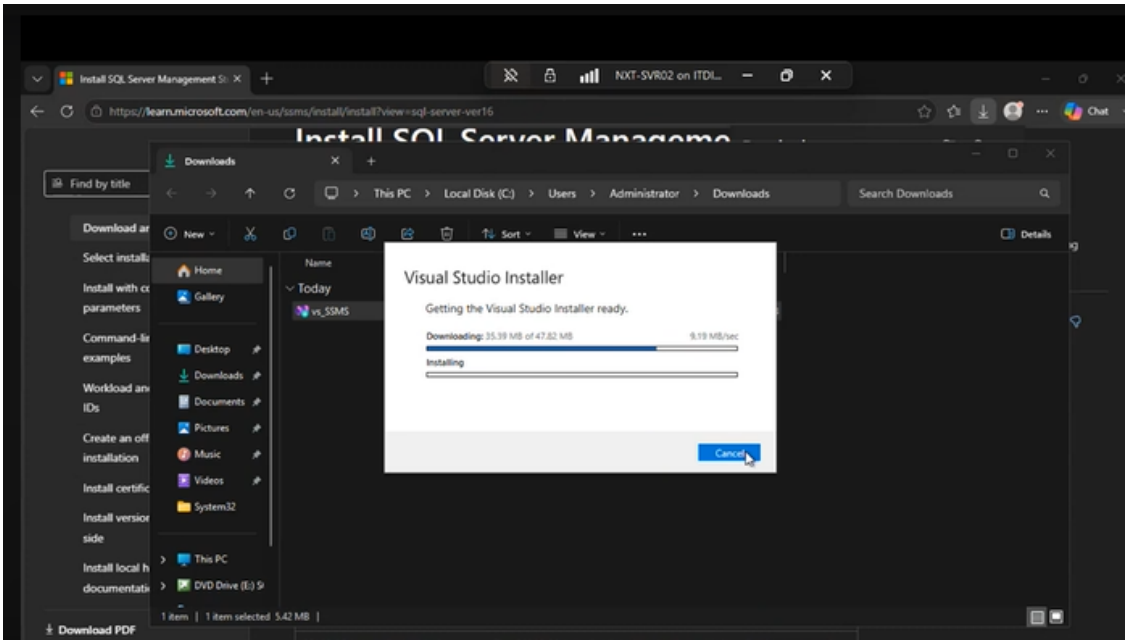
2.26 SSMS Download Page



Microsoft Learn – Install SQL Server Management Studio

On the Microsoft Learn page that opens in the browser, SSMS is described as an integrated environment used to access, configure, manage, and develop all components of your SQL infrastructure. Clicking the "Download SQL Server Management Studio 22 installer" button begins the download.

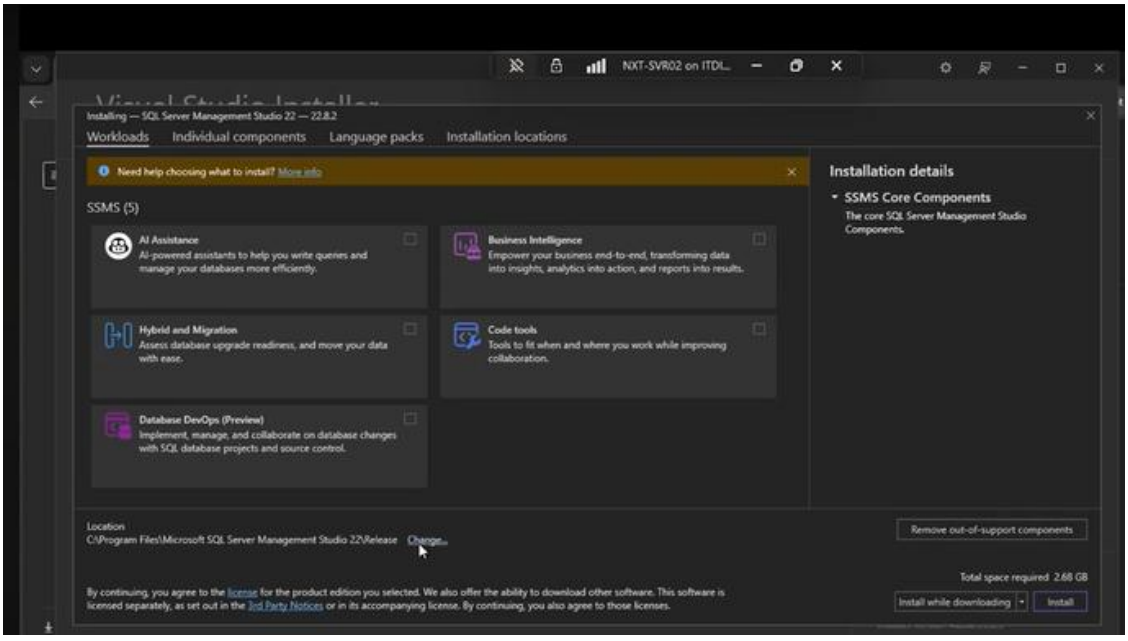
2.27 Downloading the SSMS Installer



SSMS – Visual Studio Installer Download Progress

For SSMS 22 (the latest version, built on Visual Studio Installer technology), a small bootstrapper file (vs_SSMS.exe) downloads first. It is saved to the Downloads folder, and the "Visual Studio Installer" opens and begins downloading/installing the remaining component files.

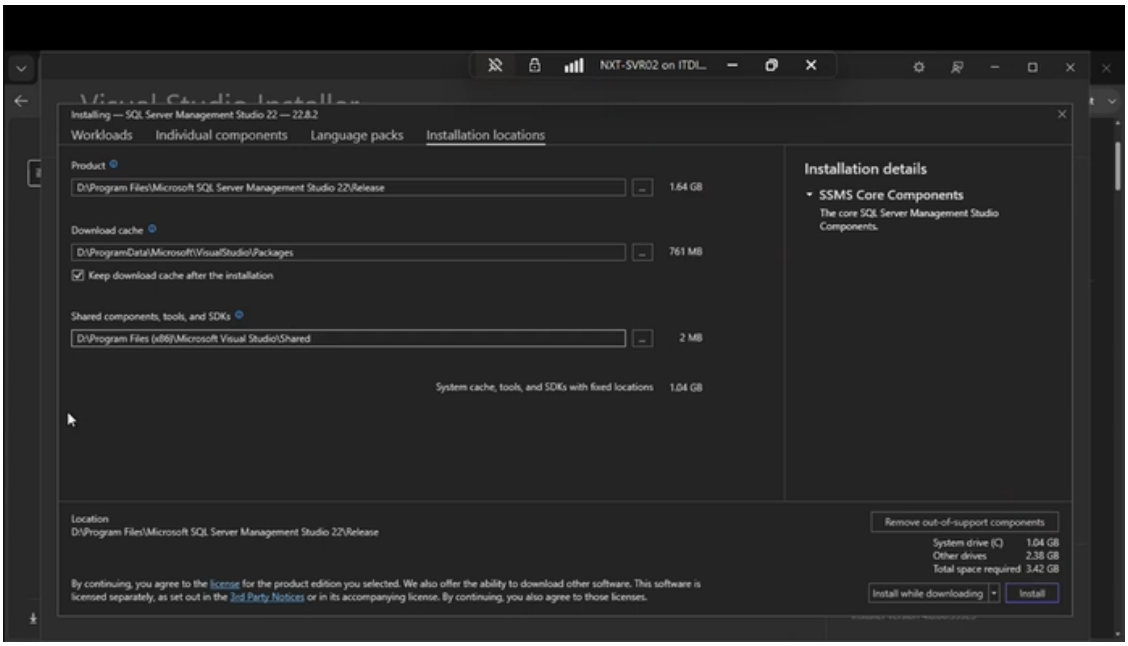
2.28 SSMS Setup — Selecting Workloads



Visual Studio Installer – SSMS Workloads

The Visual Studio Installer shows several workloads (feature sets) for SSMS 22 — AI Assistance, Business Intelligence, Hybrid and Migration, Code Tools, and Database DevOps (Preview). The "SSMS Core Components" needed for this lab (connecting to a database, writing queries, Object Explorer, basic administration) are selected by default, and below that the default install location, "C:\Program Files\Microsoft SQL Server Management Studio 22\Release," is shown — it can be changed via the "Change" link.

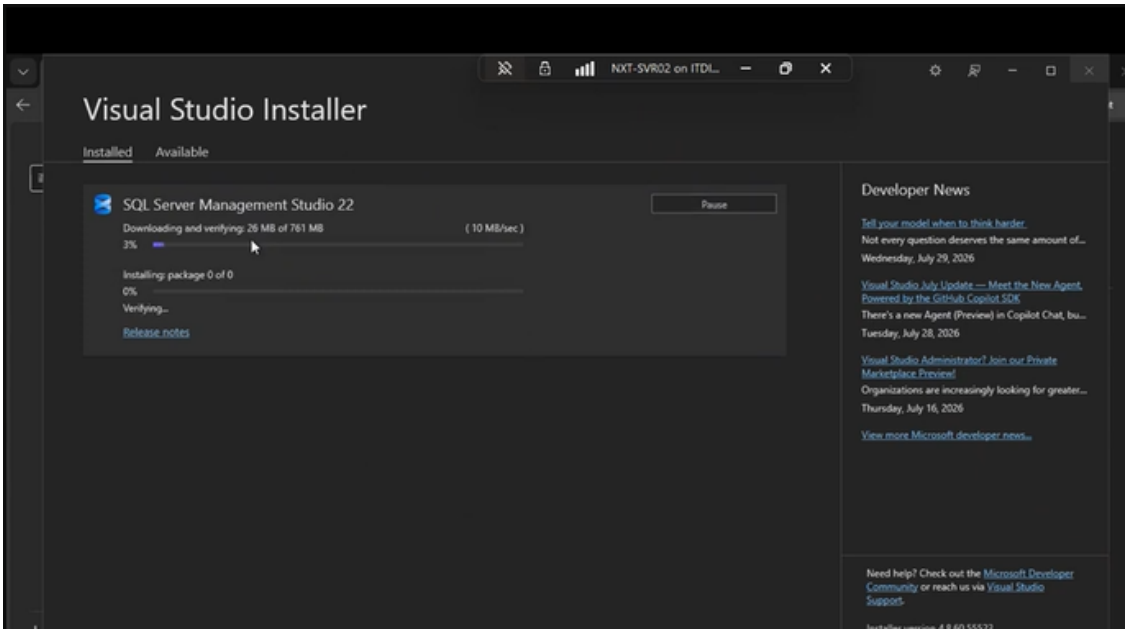
2.29 Changing the Installation Location from C: to D:



Visual Studio Installer – Installation Locations

Clicking "Change" opens the "Installation locations" tab, where all three paths — Product, Download cache, and Shared components — have been changed from the C: drive to the D: drive. This continues the same policy applied earlier in section 2.12: keeping SQL Server–related software and data off the OS drive. After agreeing to the license below, click "Install."

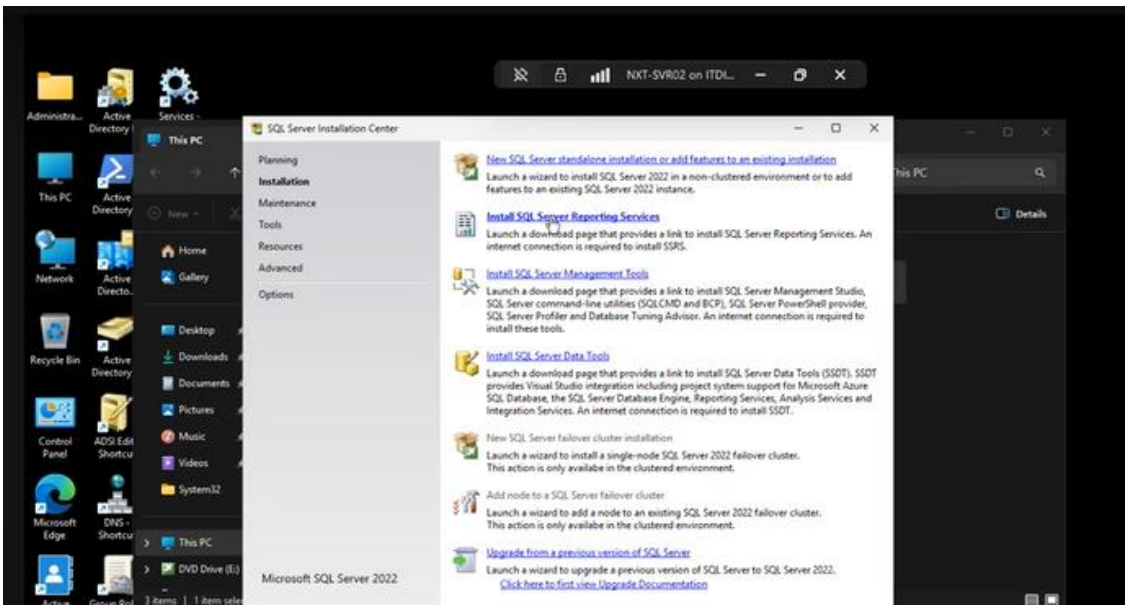
2.30 SSMS Installing



Visual Studio Installer – Installing SSMS Progress

This progress screen shows the SSMS package being downloaded, verified, and installed — the amount of data downloaded, the total size, and the download speed are all shown. Wait for this to complete before proceeding.

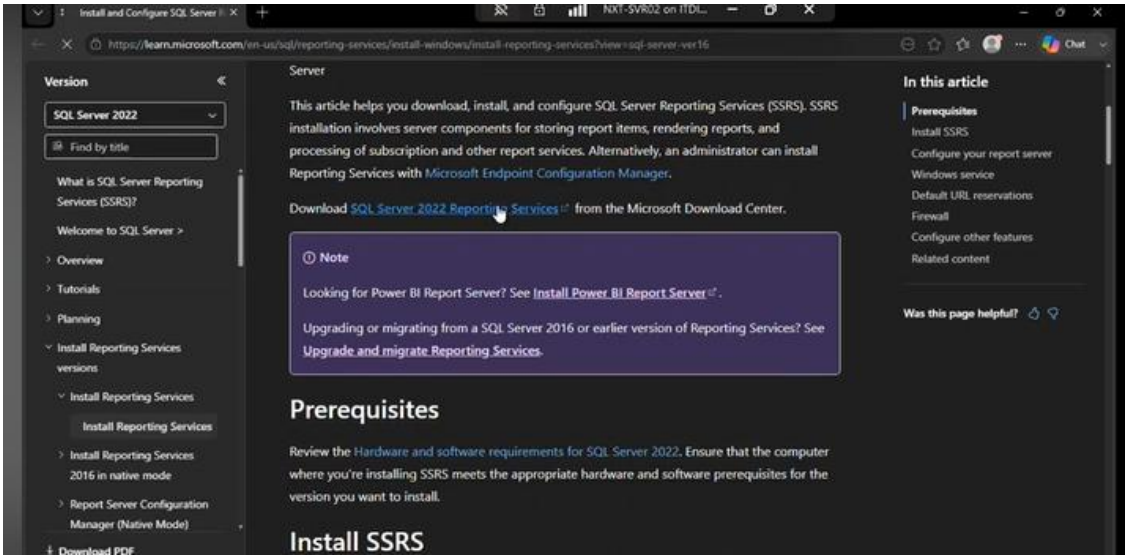
2.31 Attempting to Install SQL Server Reporting Services (SSRS)



SQL Server Installation Center – Install SQL Server Reporting Services

Once the SSMS installation completes, returning once more to the SQL Server Installation Center, this time the "Install SQL Server Reporting Services" link — the Business Intelligence tool discussed in section 1.8 — is clicked. Like SSMS, SSRS has also been a separate download/install since SQL Server 2017.

2.32 SSRS Official Documentation Page



Microsoft Learn – SSRS Documentation (reopened)

After clicking "Install SQL Server Reporting Services," the browser reopens the same Microsoft Learn documentation page seen earlier in section 1.10 — it is important to check the prerequisites and installation steps here before actually downloading SSRS.

2.33 Important Note — SSRS Cannot Be Installed on a Domain Controller

> *SQL Server Reporting Services (SSRS), along with some other SQL Server companion services, is officially "Not Supported" for installation on a server that is also functioning as an Active Directory Domain Controller. This is directly connected to the "Computer domain controller: Warning" message seen earlier during the Install Rules check in section 2.10 — a real-world problem commonly encountered. Domain Controller servers operate under much stricter security restrictions than ordinary member servers (limitations on local accounts, special service isolation rules, and so on), and these restrictions prevent SSRS from properly configuring its web service and service accounts. As a result, SSRS could not be fully installed and configured in this lab environment.*
>
> *The practical lesson here: in a real production environment, SQL Server — and especially companion services like SSRS, SSAS, and SSIS — should never be installed on a Domain Controller server. Instead, following the principle of Separation of Roles, a dedicated, non-Domain-Controller member server should be set aside for SQL Server — a principle we already discussed under "Deployment Architecture" in the "Preparation and Planning" section (1.9).*

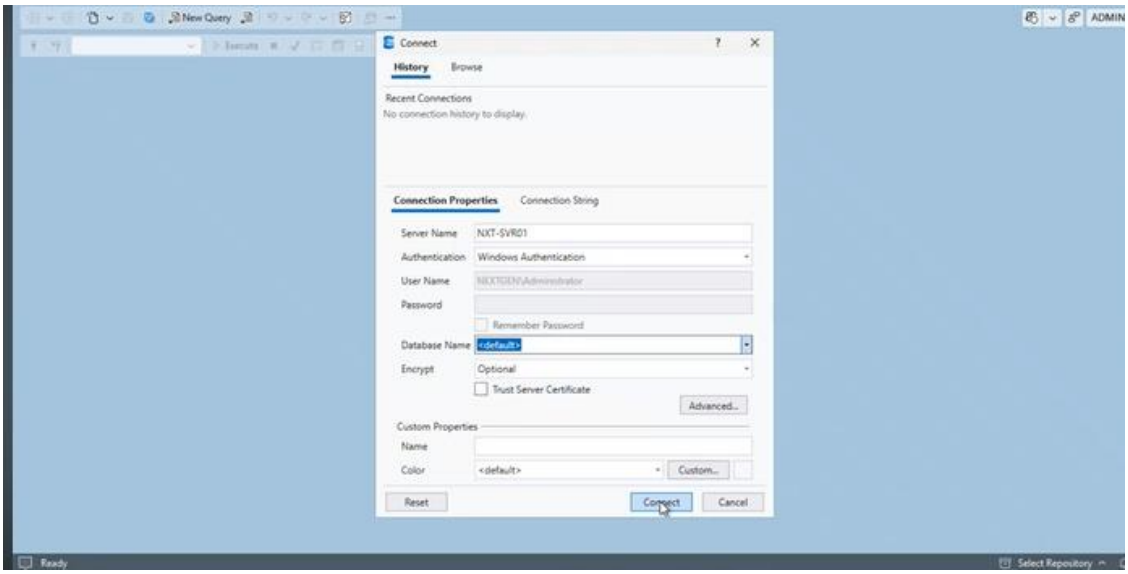
2.34 Launching SSMS for the First Time



SQL Server Management Studio 22 – splash screen

Since SSRS could not be installed at this point, the lab moves on to using SSMS to connect to the Database Engine and carry out database administration tasks instead. Opening SSMS 22 from the Desktop/Start Menu shortcut displays this splash screen.

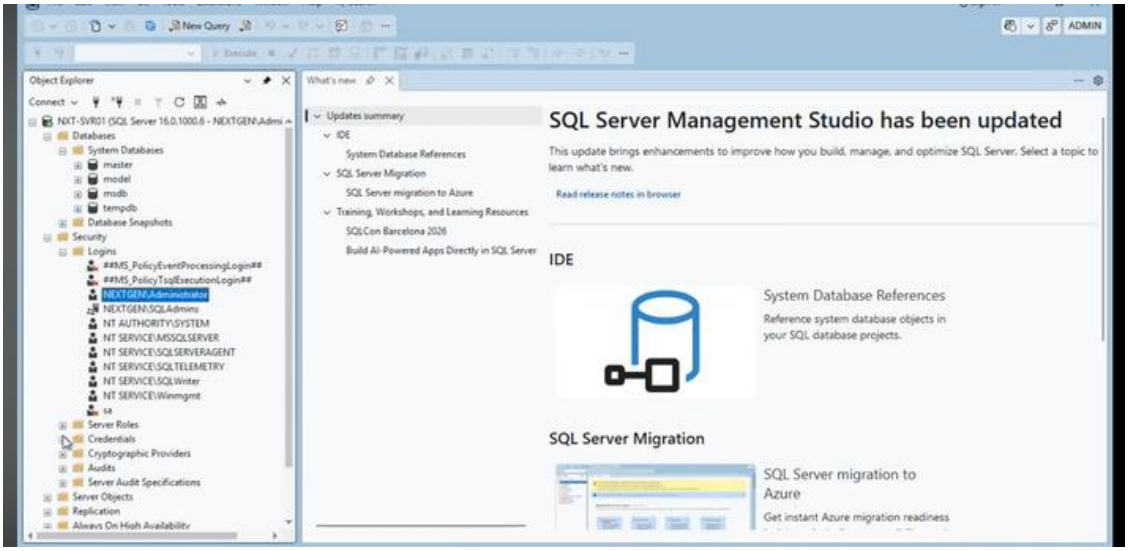
2.35 Connecting to the Server



SSMS – Connect to Server Dialog

As soon as SSMS opens, the "Connect to Server" dialog box appears automatically. Here, the "Server Name" to connect to is entered, and "Authentication" is set to "Windows Authentication" — since "Windows Authentication Mode" was chosen during SQL Server setup in section 2.17, you can connect using the domain account you are currently logged into Windows with, without re-entering a password. Clicking "Connect" connects to the SQL Server instance.

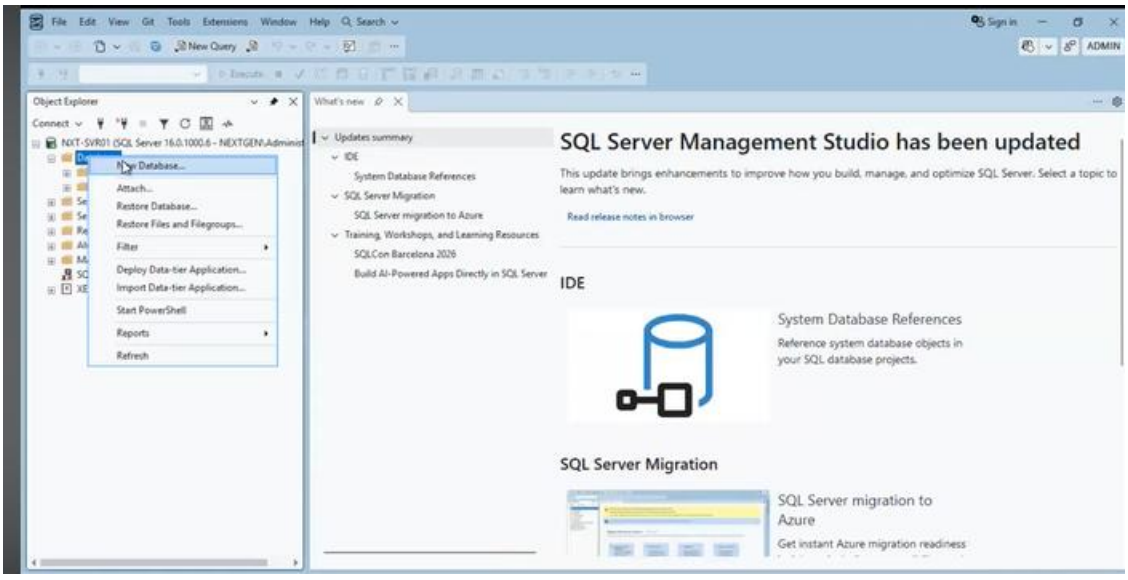
2.36 Successfully Connected — Object Explorer



SSMS – Object Explorer (Connected)

Once the connection succeeds, the "Object Explorer" panel on the left shows the entire structure of the connected SQL Server instance as a tree view. Under "Databases" → "System Databases," you can see the four default databases SQL Server automatically creates — master (stores server-level configuration data), model (used as a template when a new database is created), msdb (used by SQL Server Agent to schedule jobs/backups), and tempdb (used to store temporary objects/data). Under "Security" → "Logins," both "NEXTGEN\Administrator" and "NEXTGEN\SQLAdmins" — added as SQL Server administrators in section 2.17 — appear in the login list. This finally confirms that all the configuration decisions made from section 2.1 through 2.17 were correctly applied.

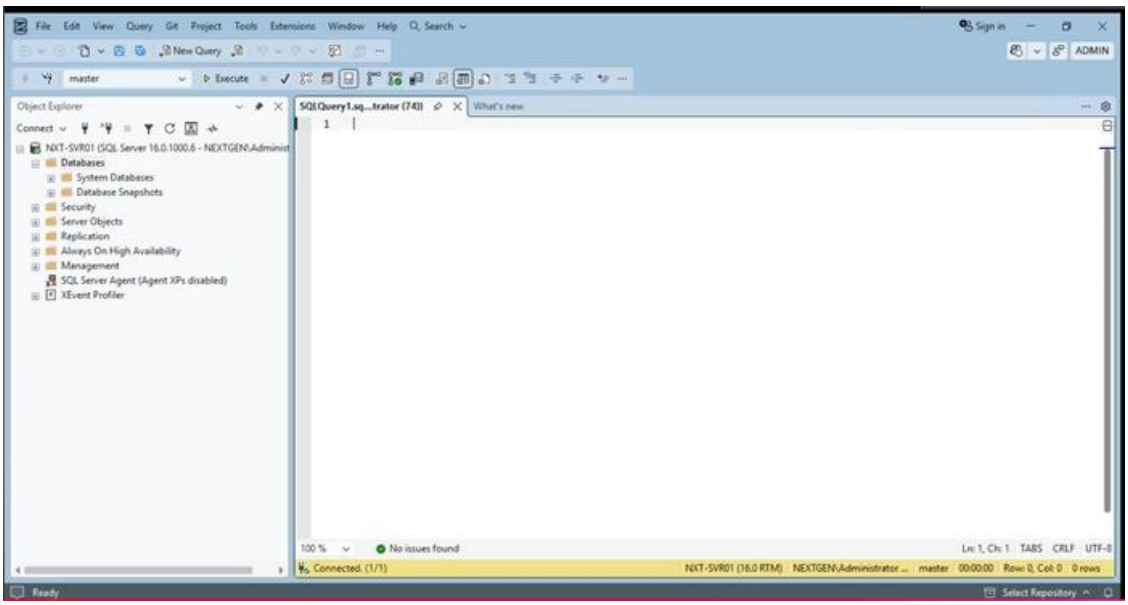
2.37 Preparing to Create a Database (GUI Method)



SSMS – Databases – Context Menu – New Database

Right-clicking the "Databases" node shows a context menu with options such as "New Database...," "Attach...," "Restore Database...," and "Restore Files and Filegroups...". Choosing "New Database..." and filling out a form is the graphical (point-and-click) way to create a database — no code needs to be written. In this lab, however, we instead create the database through T-SQL commands (writing code), which is what we cover in Parts 03/04 — both approaches achieve the same result, but creating it via code offers advantages such as being reusable as a script and easier to automate.

2.38 Opening a New Query Window



SSMS – New Query Window

Clicking the "New Query" toolbar button opens a blank T-SQL editor connected to the "master" database (as shown in the database dropdown in the toolbar). From here onward, all the SQL commands explained in Part 03 are typed into this Query window and executed.

Part 03 – 22 Essential SQL Commands Explained in Full Detail

For the lab practical, a reference guide called the "Microsoft SQL Server Basic Commands – Comprehensive Student Lab Guide," prepared specifically for beginner students, was used, containing all 22 commands shown as screenshots across the 4 pages below. First, let's look at the original pages of this reference sheet, and then go through each command individually, explaining its meaning, syntax, and practical importance in further detail.

Microsoft SQL Server Basic Commands – Comprehensive Student Lab Guide

This guide introduces essential Microsoft SQL Server commands for beginners. Each section explains the purpose of the command, syntax, and includes practical examples suitable for classroom demonstrations using SQL Server Management Studio (SSMS).

1. CREATE DATABASE

Creates a new database on the SQL Server instance.

```
CREATE DATABASE StudentDB;
GO
```

2. USE DATABASE

Changes the current context to the selected database.

```
USE StudentDB;
GO
```

3. CREATE TABLE

Creates a table with columns, data types, and constraints.

```
CREATE TABLE Students
(
    StudentID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Age INT,
    City VARCHAR(50)
);
```

4. INSERT

Adds one or more records into a table.

```
INSERT INTO Students
VALUES
(101,'John','Perera',20,'Colombo'),
(102,'Nimal','Silva',22,'Kandy'),
(103,'Kasun','Fernando',21,'Galle');
```

5. SELECT

Retrieves data from a table.

```
SELECT * FROM Students;

SELECT FirstName, City FROM Students;
```

6. WHERE

Filters rows based on conditions.

```
SELECT * FROM Students WHERE Age > 20;
SELECT * FROM Students WHERE City='Colombo';
```

7. ORDER BY

Sorts results ascending or descending.

```
SELECT * FROM Students ORDER BY Age;
SELECT * FROM Students ORDER BY Age DESC;
```

8. UPDATE

Modifies existing records.

```
UPDATE Students SET City='Kurunegala' WHERE StudentID=102;
```

9. DELETE

Removes rows from a table.

```
DELETE FROM Students WHERE StudentID=104;
```

10. COUNT

Counts matching rows.

```
SELECT COUNT(*) AS TotalStudents FROM Students;
```

11. Aggregate Functions

Performs calculations across rows.

```
SELECT AVG(Age), MAX(Age), MIN(Age) FROM Students;
```

12. LIKE

Searches using wildcard patterns.

```
SELECT * FROM Students WHERE FirstName LIKE 'J%';
```

13. IN

Matches against multiple values.

```
SELECT * FROM Students WHERE City IN ('Colombo','Kandy');
```

14. BETWEEN

Filters within a range.

```
SELECT * FROM Students WHERE Age BETWEEN 20 AND 22;
```

15. Second Table

Creates a Courses table.

```
CREATE TABLE Courses
(
 CourseID INT PRIMARY KEY,
 CourseName VARCHAR(50),
 StudentID INT
);
```

16. INNER JOIN

Combines matching rows from two tables.

```
SELECT s.FirstName,c.CourseName
FROM Students s
INNER JOIN Courses c
ON s.StudentID=c.StudentID;
```

17. GROUP BY

Groups rows before aggregation.

```
SELECT City, COUNT(*) AS Total FROM Students GROUP BY City;
```

18. VIEW

Creates a virtual table.



```
CREATE VIEW StudentView AS
SELECT StudentID,FirstName,City FROM Students;
```

19. Stored Procedure

Stores reusable SQL logic.

```
CREATE PROCEDURE GetStudents
AS
BEGIN
SELECT * FROM Students;
END;
EXEC GetStudents;
```

20. Login and User

Creates SQL login and database user (requires admin).

```
CREATE LOGIN StudentLogin WITH PASSWORD='P@ssword123';
CREATE USER StudentUser FOR LOGIN StudentLogin;
GRANT SELECT ON Students TO StudentUser;
```

21. System Functions

Returns server, database, user and date information.

```
SELECT DB_NAME();
SELECT SYSTEM_USER;
SELECT GETDATE();
SELECT @@VERSION;
SELECT @@SERVERNAME;
```

22. DROP Commands

Removes objects permanently.

```
DROP VIEW StudentView;
DROP TABLE Courses;
USE master;
DROP DATABASE StudentDB;
```

Learning Outcomes

By the end of this lab students should be able to:

- Create and manage databases
- Create and modify tables
- Insert, update, delete and retrieve data

Reference Guide – Page 4 (Commands 19-22 and Learning Outcomes)

Command 01 — CREATE DATABASE

```
CREATE DATABASE StudentDB;
GO
```

This command creates a brand-new, empty database (container) within a SQL Server instance. "GO" is not actually a real T-SQL command — it is a "batch separator" recognized by tools like SSMS/sqlcmd, telling the tool to send everything above it to the server as a single batch. Running this command requires the user to have server-level "dbcreator" or "sysadmin" permission.

Command 02 — USE DATABASE

```
USE StudentDB;
GO
```

This changes the "current database context" for the session, so that all subsequent commands run inside StudentDB rather than master. Instead of typing this, you can also select the database from the Database

dropdown in the SSMS toolbar. Forgetting the "USE" statement is one of the most common mistakes beginners make — we will see this cause a real error practically in section 4.5.

Command 03 — CREATE TABLE

```
CREATE TABLE Students
(
   StudentID INT PRIMARY KEY,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   Age INT,
   City VARCHAR(50)
);
```

This creates a new table (Students) made up of rows and columns inside a database. Inside the parentheses ( ), each column's name and data type are listed, separated by commas — INT (integer/whole numbers) and VARCHAR(50) (variable-length text up to 50 characters). The "PRIMARY KEY" constraint ensures that all values in the StudentID column are unique and can never be NULL — this makes it easy to uniquely identify each row and to build relationships between tables (see Command 16).

Command 04 — INSERT

```
INSERT INTO Students VALUES
(101,'John','Perera',20,'Colombo'),
(102,'Nimal','Silva',22,'Kandy'),
(103,'Kasun','Fernando',21,'Galle');
```

This adds one or more new rows to a table. After "VALUES," the values for a row are listed, comma-separated, inside parentheses ( ), in exactly the same column order used in the CREATE TABLE statement. As shown here, inserting 3 rows at once in a single INSERT statement (separated by commas) is more efficient than writing 3 separate INSERT statements, one per row.

Command 05 — SELECT

```
SELECT * FROM Students;
SELECT FirstName, City FROM Students;
```

This is the most widely used command in SQL, used to read data from a table. "*" means retrieving every column in the table. For both performance and clarity, in production code it is recommended to specify only the columns actually needed (e.g., FirstName, City) instead of "*" — to avoid sending unnecessary data across the network.

Command 06 — WHERE

```
SELECT * FROM Students WHERE Age > 20;
SELECT * FROM Students WHERE City='Colombo';
```

This adds a condition to a SELECT statement, filtering and retrieving only the rows that match that condition. Comparison operators (=, >, <, >=, <=, <>) can be used. Text values must be written inside single quotes when compared, while number values do not require quotes.

Command 07 — ORDER BY

```
SELECT * FROM Students ORDER BY Age;
SELECT * FROM Students ORDER BY Age DESC;
```

This sorts the result set returned by a query according to the values in a chosen column. The default sort order is ascending (from smallest to largest), and adding "DESC" changes it to descending order (largest to smallest). It is also common to use this together with WHERE.

Command 08 — UPDATE

```
UPDATE Students SET City='Kurunegala' WHERE StudentID=102;
```

This is used to modify existing row(s) in a table. "SET" specifies the new value, while "WHERE" identifies the specific row(s) to be modified.

> *An extremely important warning: if you forget the "WHERE" clause in an UPDATE statement, every single row in the table will be changed at once! It is good practice to always first run a SELECT statement with the equivalent WHERE condition before running an UPDATE, to confirm exactly which rows will be affected.*

Command 09 — DELETE

```
DELETE FROM Students WHERE StudentID=104;
```

This is used to remove row(s) from a table (the same warning about forgetting "WHERE" that applied to UPDATE applies here as well — all data in the table will be removed). DELETE operates row-by-row and is fully logged in the transaction log (so it can be rolled back). Completely different from this is the "TRUNCATE TABLE" command, which removes all data in a table at once, very quickly — but it resets identity values and generally does not support a WHERE clause.

Command 10 — COUNT

```
SELECT COUNT(*) AS TotalStudents FROM Students;
```

COUNT() is an aggregate function that counts the number of matching rows in a table. "AS TotalStudents" gives the result column a more readable name (an alias) — without it, the result column would have a default, unnamed label.

Command 11 — Aggregate Functions

```
SELECT AVG(Age), MAX(Age), MIN(Age) FROM Students;
```

Like COUNT(), functions that compute across a group of rows and return a single summary value are called "aggregate functions." AVG() = average value, MAX() = largest value, MIN() = smallest value. The widely used SUM() (total) function also belongs to this same category.

Command 12 — LIKE

```
SELECT * FROM Students WHERE FirstName LIKE 'J%';
```

This is used to search whether a text column matches a wildcard pattern. The "%" symbol represents any sequence of characters (zero or more), while "_" represents exactly one character. In the example, 'J%' matches any name starting with the letter "J" (John, Jane, ...).

Command 13 — IN

```
SELECT * FROM Students WHERE City IN ('Colombo','Kandy');
```

This checks whether a column's value matches any one of a given list of values. It is a shorter, clearer way of writing multiple OR conditions like "City='Colombo' OR City='Kandy'" — especially useful as the number of values grows.

Command 14 — BETWEEN

```
SELECT * FROM Students WHERE Age BETWEEN 20 AND 22;
```

This checks whether a column's value falls within a given range (inclusive of both the start and end values). In other words, "Age BETWEEN 20 AND 22" is equivalent to saying "Age >= 20 AND Age <= 22".

Command 15 — Second Table (Courses)

```
CREATE TABLE Courses
(
 CourseID INT PRIMARY KEY,
 CourseName VARCHAR(50),
 StudentID INT
);
```

To demonstrate a JOIN operation (Command 16) going forward, a second table (Courses) is created here. The "StudentID" column in this table conceptually represents which student a given course belongs to, referencing the Students table's StudentID column. Note: it has not been formally declared as a "FOREIGN KEY" here — in a real production database design, it is good practice to add a

"FOREIGN KEY REFERENCES Students(StudentID)" constraint on the StudentID column, to enforce referential integrity (i.e., to prevent inserting a course for a StudentID that doesn't exist).

Command 16 — INNER JOIN

```
SELECT s.FirstName, c.CourseName
FROM Students s
INNER JOIN Courses c
ON s.StudentID = c.StudentID;
```

This combines rows from two (or more) tables where the values in a shared column (here, StudentID) match, producing a single combined result set. "INNER JOIN" is the type of join that returns only the rows that match in both tables (rows that don't have a match in one of the tables are excluded from the result). "s" and "c" are "aliases" (short names) used in place of the full table names, to keep the query shorter.

Command 17 — GROUP BY

```
SELECT City, COUNT(*) AS Total FROM Students GROUP BY City;
```

This groups together rows that share the same value in a column, and then applies an aggregate function (COUNT(), AVG(), etc.) separately to each group. In the example, students are grouped by "City," and the number of students in each city is counted separately. While WHERE filters at the row level, GROUP BY is different — it combines rows together into groups.

Command 18 — VIEW

```
CREATE VIEW StudentView AS
SELECT StudentID, FirstName, City FROM Students;
```

A view is a "virtual table" based on a SELECT query, which does not store any data of its own within the database. Once created, it can be queried exactly like a normal table, e.g. "SELECT * FROM StudentView;". Benefits of using views: (1) avoiding having to rewrite a complex query used frequently, by reusing it under one name, and (2) presenting only a selected portion of the data — rather than every column/row in the underlying table — to other users, for both security and simplicity.

Command 19 — Stored Procedure

```
CREATE PROCEDURE GetStudents
AS
BEGIN
  SELECT * FROM Students;
END;
EXEC GetStudents;
```

A stored procedure is a set of SQL statements saved and stored on the database server under a name, so it can be reused (similar to the concept of a "function"/"subroutine" in other programming languages). It is created using "CREATE PROCEDURE ... AS BEGIN ... END," and can later be executed at any time simply by writing "EXEC GetStudents;" — without having to rewrite the full query. Stored procedures can also accept parameters (not shown in this simple example), which brings benefits such as centralizing business logic, increasing code reusability, and improving performance through execution plan caching.

Command 20 — Login and User

```
CREATE LOGIN StudentLogin WITH PASSWORD='P@ssword123';
CREATE USER StudentUser FOR LOGIN StudentLogin;
GRANT SELECT ON Students TO StudentUser;
```

SQL Server's security model has two levels — a "Login" (a server-level identity that grants access to connect to the entire SQL Server instance) and a "User" (a database-level identity used within a specific database). CREATE LOGIN creates a new SQL login (StudentLogin) with a password — note that since this is a "SQL Server Authentication" method, "Mixed Mode" would need to be enabled instead of the "Windows Authentication Mode" chosen in section 2.17. CREATE USER FOR LOGIN creates a user (StudentUser) within the current database (StudentDB) that maps to that login. GRANT SELECT ON ... TO ... grants that user permission only to read data (SELECT) from the Students table — not to modify, delete, or insert data. This is a practical example of the "Principle of Least Privilege" — a security principle in which each user is granted only the minimum permissions required for their role.

Command 21 — System Functions

```
SELECT DB_NAME();
SELECT SYSTEM_USER;
SELECT GETDATE();
SELECT @@VERSION;
SELECT @@SERVERNAME;
```

SQL Server provides a range of built-in functions that return metadata about the current server/database/user/date — extremely useful for diagnostics, auditing, and writing dynamic scripts:

- DB_NAME() — returns the name of the currently connected database.
- SYSTEM_USER — returns the name of the currently logged-in account.
- GETDATE() — returns the current date and time on the server.
- @@VERSION — returns the edition/version/build details of the running SQL Server.
- @@SERVERNAME — returns the server name of the current SQL Server instance.

Command 22 — DROP Commands

```
DROP VIEW StudentView;
DROP TABLE Courses;
USE master;
DROP DATABASE StudentDB;
```

"DROP" commands permanently remove database objects (views, tables, databases, etc.) — irreversibly, and without a backup they cannot be recovered — so they must be used with great care. Two important rules can be seen in this very example: first, "StudentView" (the view created on the Students table in Command 18) is dropped before "Courses," because attempting to drop a table while dependent objects still exist on it often results in an error (dependent objects must be dropped first). Second, "USE master;" is used to switch the database context before writing "DROP DATABASE StudentDB;" — because SQL Server does not allow you to drop a database while you are currently "inside" it (i.e., while it is your current context via USE) — this connects directly back to the USE concept from Command 02.

Learning Outcomes — What This Lab Achieves

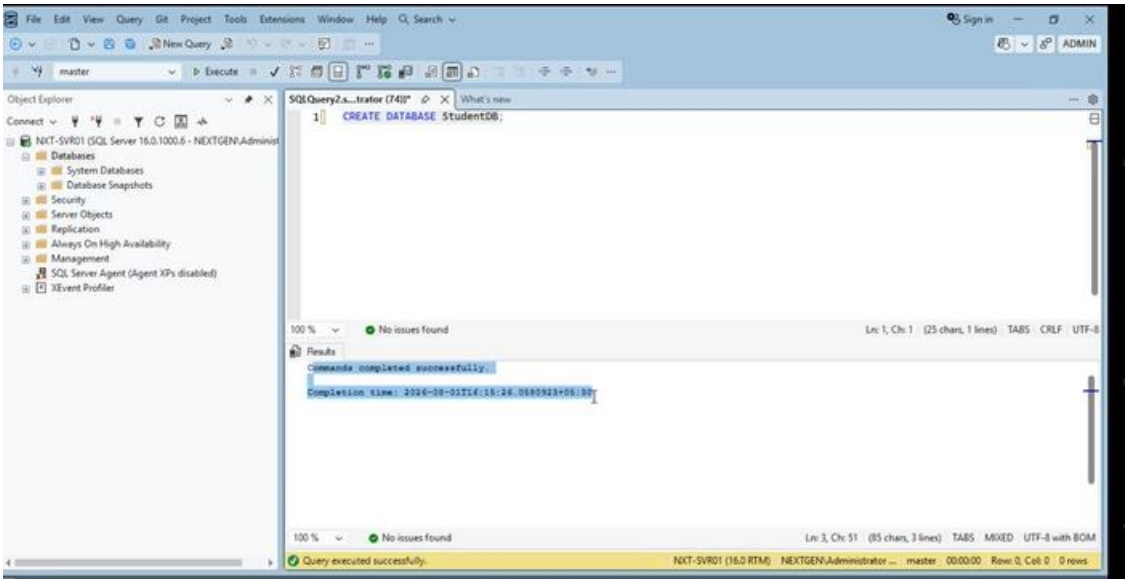
After fully and practically following the 22 SQL commands above, a student should be able to:

- Create and manage databases and tables
- Modify table structure and use constraints (such as Primary Key)
- Insert, update, delete, and retrieve data
- Filter and sort data using WHERE, ORDER BY, LIKE, IN, and BETWEEN
- Summarize data using aggregate functions such as COUNT, AVG, MAX, MIN, and GROUP BY
- Combine and query data across multiple tables using INNER JOIN
- Create and use views and stored procedures
- Create logins/users and manage permissions using GRANT
- Retrieve server/database metadata using system functions
- Safely remove objects in the correct order using DROP commands

Part 04 – Running the Commands Practically in SSMS

Let's now walk step by step through how all the theory above was actually typed and executed in the New Query window (section 2.38), based on the screenshots taken — this includes not only successful outcomes, but also two real errors commonly encountered by beginners, which offer valuable lessons of their own.

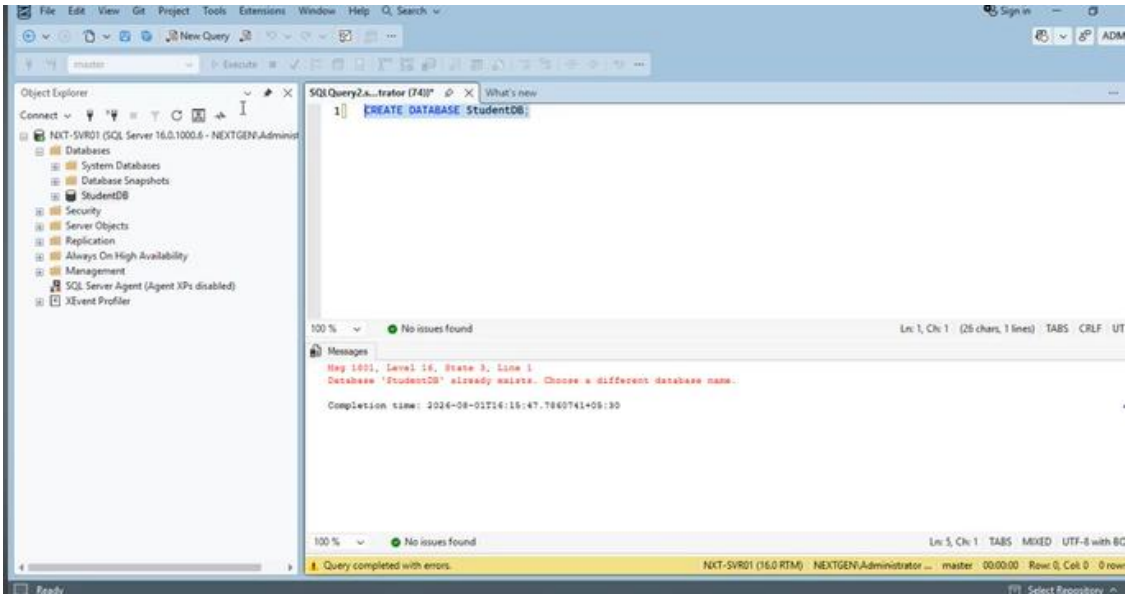
4.1 Running the CREATE DATABASE Command



SSMS – CREATE DATABASE StudentDB; executed

In the Query window, "CREATE DATABASE StudentDB;" was typed and run using "Execute" (or the F5 key). In the "Results" panel below, the message "Commands completed successfully." appears, along with the exact completion time — confirming that the database "StudentDB" was successfully created.

4.2 Attempting to Re-create an Already-Existing Database — an Error

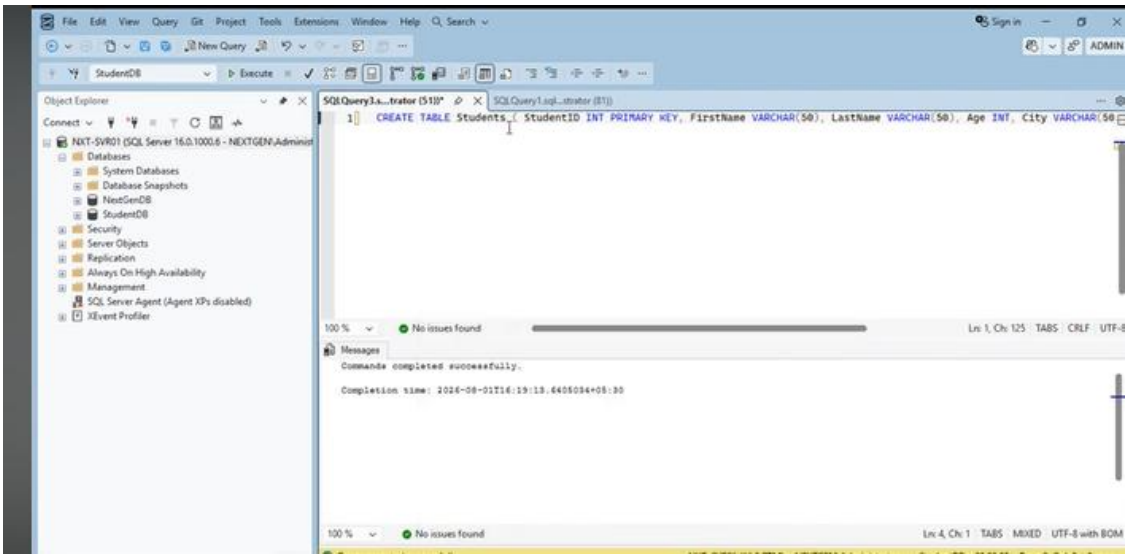


SSMS – Database already exists Error

Running the same command again (CREATE DATABASE StudentDB;) produces the error: "Msg 1801, Level 16, State 3, Line 1 — Database 'StudentDB' already exists. Choose a different database name." This is expected, safe behavior in SQL Server — since two databases cannot share the same name, SQL Server simply rejects the operation without causing any harm to the existing database. On

the left, "StudentDB" can already be seen listed under the "Databases" node in Object Explorer — further proof that the creation in section 4.1 was successful.

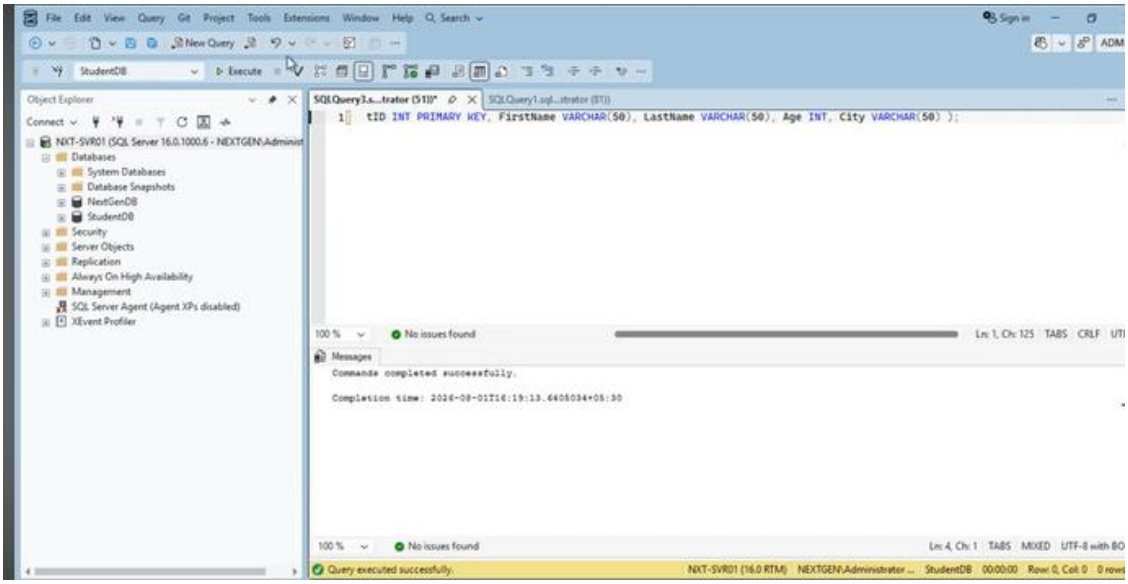
4.3 Running the CREATE TABLE Students Command



SSMS – CREATE TABLE Students executed

Notice that from this point on, the Database dropdown in the toolbar has been changed to "StudentDB" (in line with Command 02, USE DATABASE) — confirming that subsequent commands will run inside StudentDB. After running the "CREATE TABLE Students (...)" command, the "Commands completed successfully." message appears again. (You may also notice a database called "NextGenDB" appears in Object Explorer — this is a separate, pre-existing database on this instance used for other lab/testing purposes, and is not related to this exercise.)

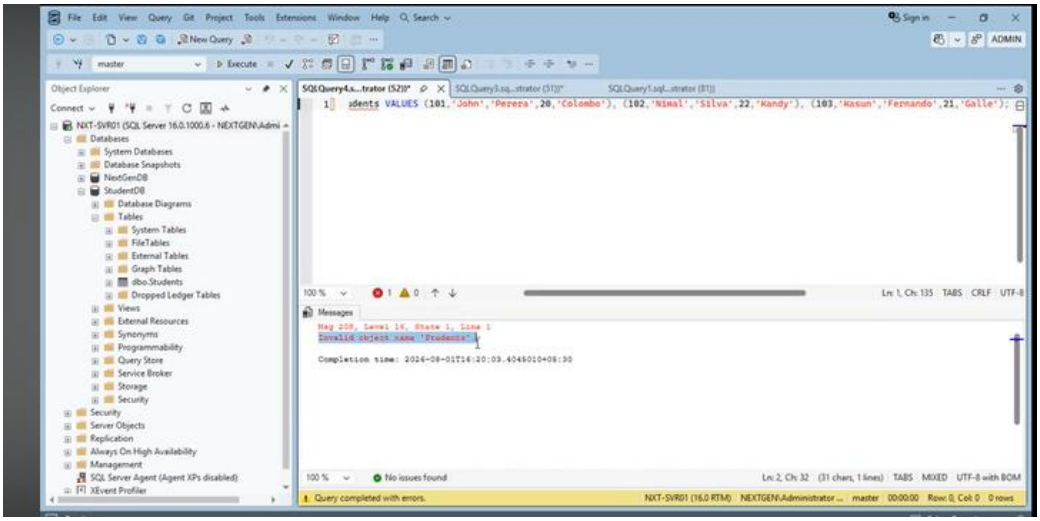
4.4 Scrolling to View the Full CREATE TABLE Statement



SSMS – remaining portion of the CREATE TABLE statement

Scrolling horizontally within the query editor reveals the remaining column definitions of the long CREATE TABLE statement (Age INT, City VARCHAR(50)). The same "Commands completed successfully." message appears again for this query tab — confirming the statement executed correctly and in full, with no truncation.

4.5 An Error With the INSERT INTO Command — an Important Lesson



SSMS – Invalid object name Students Error

This is the most important "teaching moment" in the entire lab practical. Running "INSERT INTO Students VALUES (...)" produces the error: "Msg 208, Level 16, State 1, Line 1 — Invalid object name 'Students'." — yet on the left, Object Explorer clearly shows the "StudentDB → Tables → dbo.Students" table listed! Why does this happen?

The reason is this: in the query tab where this command was typed (SQLQuery4), the Database dropdown in the toolbar shows "master" — not "StudentDB"! This is a different query tab (SQLQuery4) from the one where CREATE TABLE was run in sections 4.3/4.4 (SQLQuery3) — SSMS allows multiple query tabs to be open at once, and each tab can have its own separate database context. SQL Server looks for a table called "Students" inside the "master" database, doesn't find one there, and returns the error.

There are two ways to fix this: (1) add "USE StudentDB;" as the first line in this same query tab before the INSERT statement, or (2) select "StudentDB" from the Database dropdown in the toolbar before executing. This error reinforces once again the importance of Command 02 (USE DATABASE) — it is one of the most commonly encountered practical lessons for anyone working with SQL Server, about the importance of correctly selecting the "current database context."